

THE COMPLETE GUIDE

```
g = StateGraph(State)
g.add_node("agent", fn)
g.add_edge(START, "agent")
g.compile(checkpointer)
```

Mastering LangGraph & AI Agents

From the durable-state-machine mental model to production-grade agents — the edge cases, the patterns, and the intuition. Current to LangGraph 1.2 and LangChain 1.0.

StateGraph

Reducers

the Send API

Checkpointers

interrupt()

Command

create_agent

Middleware

Multi-Agent

Streaming

Durable Execution

Time Travel

A structured path · 28 chapters · 4 parts · current to the 1.x era

Contents

Four parts, twenty-eight chapters — foundations through mastery

PART I — FOUNDATIONS & INTUITION

1. What LangGraph Actually Is

- The one-sentence intuition
- Why a graph, and not a chain?
- Agents vs. workflows: the spectrum, not the dichotomy
- The execution model: super-steps (the Pregel/BSP heritage)
- Where LangChain fits in 1.0
- What you actually install

2. The Agent Loop, From First Principles

- What an "AI agent" really is
- Seeing the loop as a graph
- The three "ways" to build an agent in LangGraph
- Why the loop needs a budget

3. State Is the Whole Game

- The central intuition
- Defining state: the three schema styles (a "ways" decision)
- The partial-update model, and why it matters
- Input, output, and private channels
- The mental model: state as a set of named channels

4. Reducers & the Super-Step

- The problem reducers solve
- Attaching a reducer with Annotated
- The most important reducer: `add_messages`
- When do you actually need a reducer? (intuition)
- The subtle edge cases of reducers

5. Nodes, Edges & Control Flow

- Nodes: where work happens
- The Runtime object: context without globals
- Edges: the three kinds
- Cycles: the feature that makes agents possible
- Termination and the recursion limit (edge-case preview)
- Putting it together: the canonical agent graph by hand

PART II — THE CORE MACHINERY

6. Command: Update State and Route in One Move

- The intuition
- Command vs. conditional edges: the "ways" decision

Navigating to a parent graph

7. The Send API & Map-Reduce

The problem: dynamic, data-dependent parallelism

How Send works

The intuition

Edge cases that bite

8. Persistence & Checkpointers

The intuition: a save point after every step

Threads: the unit of persistence

Choosing a checkpointer (the "ways")

What a checkpoint costs, and DeltaChannel

The read-execute-write cycle

9. Time Travel: get_state, update_state & Replaying

The intuition

Inspecting the current and past states

Editing state mid-run with update_state

Forking: exploring an alternate timeline

Replaying vs. resuming (a distinction that matters)

10. Human-in-the-Loop: interrupt() Deeply

Why human-in-the-loop is a persistence feature

The interrupt() function

THE edge case: the node re-runs from the top on resume

THE second edge case: never wrap interrupt() in try/except

Static vs. dynamic interrupts (the "ways")

Resuming multiple parallel interrupts

The "never resumes" case

The legacy pattern: NodeInterrupt

How agents get human-in-the-loop for free

11. Streaming

Why streaming is not optional

The stream modes (a "ways" decision you'll make constantly)

Streaming custom progress from inside a node

astream_events and the v3 API

12. Memory: Short-Term, Long-Term & the Store

Two kinds of memory, two mechanisms

The Store API

How agents use long-term memory

Edge cases and discipline

PART III — BUILDING AGENTS

13. The Prebuilt Agent: create_agent

Start here, always

The parameters that matter
Structured output, the modern way
When `create_agent` is enough — and when it isn't

14. Middleware: The Modern Control Surface

The intuition
The lifecycle hooks
Writing custom middleware
ModelRequest and ModelResponse
Why middleware beats the old way

15. The Built-in Middleware Catalog

Context management
Safety and guardrails
Reliability
Capability and structure
Composing them: a production agent

16. Tools & Tool Calling

What a tool is
Binding tools and the ToolNode
Tools that drive control flow: returning Command
Tool-calling edge cases

17. Custom Agents with StateGraph

When to drop down
Building ReAct by hand (the reference implementation)
Mixing deterministic and agentic — the LangGraph superpower

18. Multi-Agent Architectures

Why multiple agents
The architectures (the "ways")
Implementing handoffs with Command
Subgraphs as agents
State sharing: the central design decision

19. Subgraphs

The intuition
The key question: shared keys vs. transformed state
Streaming and persistence through subgraphs

PART IV — EDGE CASES, RELIABILITY & PRODUCTION

20. Reducer & State Edge Cases

21. Interrupt & Resume Edge Cases

22. Failure, Retries & Durable Execution

Node retries with RetryPolicy
Per-node timeouts (1.2)

Node-level error handlers and the Saga pattern (1.2)
Graceful shutdown (1.2)
The boundary problem: graph durability vs. workflow durability

23. Performance & Cost

Checkpoint overhead and DeltaChannel
Node-level caching
Context window management is cost management
Parallelism, with a leash
Model selection per step
Reliability features double as performance features

24. Recursion, Loops & Termination

The recursion limit, precisely
Designing real stopping conditions
Termination edge cases

25. Debugging & Observability

LangSmith: tracing built for agents
Stream-based and state-based inspection
Visualizing the graph
Decoding the common errors

26. Testing Agents

Test nodes in isolation
Mock or emulate the model
Test the durability features
Evaluation, not just assertion

27. Common Bugs & Anti-Patterns

28. Best Practices & Mental Models Reference

The core proverbs
Decision: agent vs. workflow
Decision: prebuilt vs. custom vs. functional
Decision: checkpointer vs. store
Decision: conditional edges vs. Command
Decision: which interrupt
The pre-production checklist
The toolchain

How to Use This Guide

This book is built around three promises in your request: *edge cases*, *ways*, *intuition*. Every chapter is written to serve all three.

- **Intuition first.** Before any API, you get the mental model — what the machinery *is*, why it exists, and what problem it quietly solves. If you only remember the intuition, you can re-derive the API.
- **The "ways."** LangGraph almost always gives you more than one route to a result: a prebuilt agent, a custom graph, or the functional API; a conditional edge or a `Command`; a static interrupt or a dynamic one. Each chapter names the alternatives and tells you which to reach for and when.
- **Edge cases as first-class content.** The difference between a demo and a deployable agent is the edge cases: a node that re-runs and fires a side effect twice, two parallel branches that clobber the same state key, an in-memory checkpointer that forgets everything on restart. These are not footnotes here; whole chapters are devoted to them.

The guide is current to the **1.x era**: LangGraph 1.2 and LangChain 1.0, where `langgraph.prebuilt` gave way to `langchain.agents`, middleware became the primary control surface for agents, and durable execution (per-node timeouts, error handlers, graceful drain) landed as a first-class concern.

The four parts:

- **Part I — Foundations & Intuition.** What LangGraph really is, the agent loop from first principles, and the three things everything else is built on: state, reducers, and control flow.
- **Part II — The Core Machinery.** `Command`, the `Send` API, persistence, time travel, human-in-the-loop, streaming, and memory.
- **Part III — Building Agents.** The prebuilt agent, middleware, tools, custom graphs, multi-agent systems, and subgraphs — the practical "ways" to assemble an agent.
- **Part IV — Edge Cases, Reliability & Production.** The failure modes, the durability features, performance, termination, debugging, testing, and a hard-won catalog of bugs and best practices.

A note on philosophy. LangGraph's core insight is deceptively small: *an agent run is a state machine, and that state should be durable*. A long workflow should survive a server restart, a human can step in mid-run, and you can rewind to any past checkpoint. Hold that idea — durable state machine — and the rest of the framework stops feeling like a grab-bag of features and starts feeling inevitable.

"LangChain is the fastest way to build an agent; LangGraph is the way to make it controllable and durable." Start high-level, drop down when you need control. They are the same runtime — agents built with LangChain's `create_agent` are LangGraph graphs underneath.

Part I — Foundations & Intuition

The mental models that make everything else obvious. We build up from "what is this thing" to the three pillars — state, reducers, control flow — that the entire framework rests on.

1. What LangGraph Actually Is

The one-sentence intuition

LangGraph is a runtime for building agents and workflows as durable state machines. You describe your application as a graph: nodes do work, edges decide what runs next, and a shared **state** object flows through. The runtime executes that graph, and — this is the part that matters — it can checkpoint the state after every step so the whole thing can pause, resume, recover from a crash, or rewind in time.

Everything else is detail in service of that sentence.

Why a graph, and not a chain?

The first generation of LLM tooling modeled applications as **chains**: linear pipelines, $A \rightarrow B \rightarrow C$. Chains are wonderful until your application needs to do something a straight line can't:

- **Loop.** An agent calls a tool, looks at the result, and decides whether to call another. That's a cycle, and a DAG (directed *acyclic* graph) forbids cycles by definition.
- **Branch on content.** "If the user is angry, route to the escalation path." A chain has no natural place for that decision.
- **Pause.** "Wait for a human to approve this refund before continuing." A chain has no notion of stopping and resuming hours later.
- **Run things in parallel and recombine.** Fan out to five researchers, then merge their findings.

LangGraph models your application as a **cyclic graph** precisely because agentic control flow is cyclic. The loop — *think, act, observe, repeat* — is the fundamental shape of an agent, and the graph is the data structure that represents it honestly.

Agents vs. workflows: the spectrum, not the dichotomy

People ask "is this an agent or a workflow?" as if it's binary. It's a spectrum of **who decides the control flow**:

- **Workflow:** *you* decide the path at design time. The graph's edges are fixed; the LLM fills in content at each node but never chooses what runs next. Predictable, testable, cheap.
- **Agent:** *the LLM* decides the path at runtime. It picks tools, decides when it's done, and the control flow emerges from its choices. Flexible, powerful, harder to bound.

The intuition that will save you money and grief: **most problems want a workflow with agentic pieces, not a fully autonomous agent.** Give the LLM authority only where the decision genuinely needs its judgment. LangGraph is unusual among agent frameworks precisely because it lets you place that dial anywhere — deterministic edges where you want control, LLM-chosen routing where you want flexibility, in the same graph.

The execution model: super-steps (the Pregel/BSP heritage)

Here's the mental model for *how* a graph runs, and it explains a surprising amount of behavior.

LangGraph executes in **super-steps**, borrowed from Google's Pregel graph-processing model and the Bulk Synchronous Parallel paradigm. A super-step is one "tick" of the graph:

1. All nodes that are ready to run in this tick execute — **potentially in parallel**.
2. Each node returns its updates to the state.
3. The runtime **applies all those updates at once** to produce the new state.
4. The runtime figures out which nodes are ready next, and the next super-step begins.

Two consequences follow directly, and they're the root of half the "why did my graph do that?" questions:

- **Nodes in the same super-step don't see each other's writes.** They all read the state as it was at the *start* of the tick. If `node_a` and `node_b` run in parallel, `node_b` cannot see what `node_a` just wrote — those writes haven't been applied yet.
- **When two parallel nodes write the same key, the runtime needs a rule to combine them.** That rule is a **reducer** (Chapter 4). Without one, you get an `InvalidUpdateError`. This isn't a bug; it's the BSP model refusing to silently pick a winner.

A checkpoint is saved at the boundary between super-steps. That's the granularity of "pause and resume" — you resume from the start of a tick, never from the middle of a node. Tuck that away; it becomes load-bearing when we reach interrupts (Chapter 10).

Where LangChain fits in 1.0

As of the 1.0 releases (October 2025), the division of labor is clean:

- **LangChain** is the high-level layer: `create_agent`, model integrations, the standard tool-calling loop, and **middleware**. It's the fastest way to ship a capable agent.
- **LangGraph** is the low-level runtime: `StateGraph`, persistence, streaming, human-in-the-loop, durable execution. It's what you drop down to when you need fine-grained control.

Crucially, **they are not competitors** — `create_agent` *builds a LangGraph graph*. You can start with the high-level agent and, when you hit its ceiling, compose it into a larger custom graph without a rewrite. The one migration note from the 1.0 cutover: `langgraph.prebuilt` (home of the old `create_react_agent`) is deprecated, and its enhanced successor lives in `langchain.agents`.

What you actually install

```
pip install -U langgraph          # the runtime (1.2+)
pip install -U langchain          # high-level agents + middleware (1.0+)
pip install -U langchain-openai langchain-anthropic # model providers
# Production persistence:
pip install -U langgraph-checkpoint-postgres
```

You will reach for `langgraph` directly the moment you need a custom graph, and `langchain.agents` when a prebuilt agent will do. This book teaches both, because mastery means knowing which one a given problem deserves.

2. The Agent Loop, From First Principles

What an "AI agent" really is

Strip away the hype and an LLM agent is a strikingly simple loop:

1. Put the user's request and a list of available **tools** in front of the model.
2. The model either **answers** (it's done) or **requests a tool call** (it needs information or an action).
3. If it requested a tool, you **execute** that tool and feed the result back.
4. **Go to step 2.**

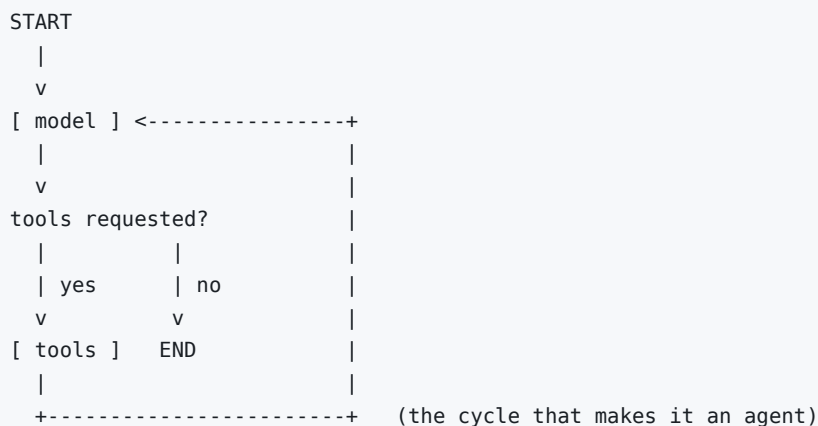
That's it. The "intelligence" is the model deciding, each time around, whether it has enough to answer or needs to act again. This loop has a name in the literature — **ReAct** (Reason + Act) — and essentially every tool-using agent is a variation on it.

The reason agents feel hard is not the loop; it's everything *around* the loop: keeping the conversation in bounds, stopping when the model goes in circles, persisting state so a long task survives a restart, letting a human intervene, recovering when a tool fails. LangGraph exists to handle that "everything around the loop" so the loop itself stays simple.

Seeing the loop as a graph

Map the four steps onto a graph and the structure is immediate:

- A **model node**: calls the LLM, gets back either a final answer or tool calls.
- A **tool node**: executes whatever tools the model asked for.
- A **conditional edge** out of the model node: *if* the model requested tools, go to the tool node; *else*, go to **END**.
- A **plain edge** from the tool node back to the model node — the cycle that makes it an agent.



That diagram is the skeleton of every agent you will ever build. When you call `create_agent`, this is the graph it constructs for you. When you build a custom agent, this is the graph you assemble by hand. Internalize it once and prebuilt vs. custom stops being mysterious — it's the same shape, built at two different altitudes.

The three "ways" to build an agent in LangGraph

This is your first big fork, so let's name all three routes up front. You'll meet each in depth later; for now, build the map.

Way 1 — The prebuilt agent (`create_agent`). One function call gives you the full ReAct loop, plus structured output, memory, and a middleware system for customization. Reach for this *first*, always. It covers the large majority of real agents.

```
from langchain.agents import create_agent

agent = create_agent(
    model="anthropic:claude-sonnet-4-5",
    tools=[search, calculator],
    system_prompt="You are a helpful research assistant.",
)
result = agent.invoke({"messages": [("user", "What's the GDP of France divided by its population?")]})
```

Way 2 — A custom `StateGraph`. You define the nodes, edges, and state yourself. Reach for this when you need control the prebuilt agent doesn't expose: a bespoke routing decision, a deterministic step wired between agentic ones, multiple cooperating sub-agents, custom state beyond the message list.

```
from langgraph.graph import StateGraph, MessagesState, START, END

builder = StateGraph(MessagesState)
builder.add_node("model", call_model)
builder.add_node("tools", ToolNode(tools))
builder.add_edge(START, "model")
builder.add_conditional_edges("model", should_continue) # -> "tools" or END
builder.add_edge("tools", "model")
graph = builder.compile()
```

Way 3 — The functional API (`@entrypoint` / `@task`). You write ordinary Python control flow — `if`, `for`, `while` — and decorate functions to get durability, checkpointing, and human-in-the-loop *without* drawing a graph. Reach for this when your logic is naturally imperative and a graph feels like ceremony, but you still want LangGraph's persistence.

```
from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import InMemorySaver

@task
def call_model(messages): ...

@entrypoint(checkpointer=InMemorySaver())
def agent(messages):
    while True:
        response = call_model(messages).result()
        if not response.tool_calls:
            return response
        messages += run_tools(response.tool_calls)
```

The intuition for choosing: start at Way 1. Drop to Way 2 when you need to *shape the control flow*. Choose Way 3 when the control flow is naturally imperative and you'd rather write Python than wire a graph. They are interoperable — a `@task` can call a graph; a graph node can call an agent.

Why the loop needs a budget

A subtle but critical point that previews Chapter 24: the agent loop can run forever. The model can keep requesting tools, observing results, and requesting more, never deciding it's done. Every LangGraph execution therefore carries a **recursion limit** (default 25 super-steps); exceed it and you get a `GraphRecursionError`. That limit is a seatbelt, not a strategy — a well-designed agent has a real stopping condition (a step budget, a tool-call cap, a "you must answer now" nudge). Treat an agent that regularly hits the recursion limit as a design smell, not a number to raise.

3. State Is the Whole Game

The central intuition

If you understand state, you understand LangGraph. **State is the shared memory that flows through your graph** — the single object every node reads from and writes to. The graph's structure (nodes and edges) is just plumbing; the *state* is the thing that actually carries meaning from step to step.

Three ideas make state click:

1. **State is a schema.** You declare its shape up front — what keys exist and what types they hold.
2. **Nodes return partial updates, not the whole state.** A node returns a dict of *only the keys it changed*. The runtime merges that into the full state for you.
3. **How keys merge is configurable** — by default, last-write-wins (overwrite); with a reducer, something smarter (append, union, custom).

Defining state: the three schema styles (a "ways" decision)

LangGraph accepts three Python constructs for your state schema, and the choice has real consequences.

```
from typing import TypedDict
from pydantic import BaseModel
from dataclasses import dataclass

# Way 1: TypedDict — most common, zero runtime overhead, no validation
class StateA(TypedDict):
    user_input: str
    response: str

# Way 2: Pydantic BaseModel — runtime validation of inputs, at a cost
class StateB(BaseModel):
    user_input: str
    response: str = ""

# Way 3: dataclass — mutable, attribute access, lightweight
@dataclass
class StateC:
    user_input: str
    response: str = ""
```

The intuition for choosing: Use **TypedDict** by default — it's a type hint, erased at runtime, so it's free. Reach for **Pydantic** when you want the framework to *validate* state at the boundary (reject a malformed input before it corrupts a run) and you'll pay a small per-update validation cost. Use a **dataclass** when you like attribute access (`state.user_input`) and want defaults without Pydantic's weight. Most production graphs are TypedDict; Pydantic earns its place at untrusted input boundaries.

The partial-update model, and why it matters

This trips up everyone once. A node does **not** mutate the state and does **not** return the whole state. It returns a small dict of changes:

```
class CounterState(TypedDict):
    count: int
    label: str

def node_a(state: CounterState) -> dict:
    return {"count": 5, "label": "from A"}  # changed both

def node_b(state: CounterState) -> dict:
    return {"count": 10}                  # changed ONLY count
```

After `node_a` then `node_b` run in sequence, `count` is `10` (B overwrote it) but `label` is still `"from A"` — because `node_b` didn't mention `label`, the runtime left it untouched. **Keys you don't return are preserved; keys you do return are merged in.** Returning the whole state object "to be safe" is an anti-pattern: it's noise, and it can accidentally clobber keys other nodes own.

***Edge case — never mutate state in place.** Writing `state["count"] += 1` inside a node and returning nothing is a bug. With checkpointing, the runtime tracks updates by what you return; an in-place mutation may be invisible to the persistence layer, may not trigger reducers, and breaks time-travel. Always return a dict of changes. Treat the incoming `state` as read-only.*

Input, output, and private channels

Your graph's *internal* state often has more keys than you want to expose. LangGraph lets you separate the public contract from the internal scratch space:

```
class InputState(TypedDict):
    user_input: str

class OutputState(TypedDict):
    answer: str

class OverallState(TypedDict):  # everything, including internal scratch
    user_input: str
    scratch: str
    answer: str

builder = StateGraph(OverallState, input_schema=InputState, output_schema=OutputState)
```

Now callers pass only `user_input` and receive only `answer`; the messy middle (`scratch`) stays private. This is the clean way to keep an agent's interface small while its internals stay rich. You can go further and define a fully private channel that only a couple of nodes share, invisible even to the overall state — useful for passing data between two specific nodes without polluting the global namespace.

The mental model: state as a set of named channels

LangGraph internally calls each state key a **channel**. A channel is a named slot that holds a value, can be read or written by nodes, and has an associated rule (its reducer) for how writes combine. "State" is just the collection of channels. This vocabulary matters because some advanced features (custom channel types, the `DeltaChannel` that cuts checkpoint size for long threads) operate at the channel level. When you hear "channel," think "one key of the state, plus its merge rule."

4. Reducers & the Super-Step

The problem reducers solve

Recall the super-step model: nodes in the same tick run in parallel and their updates are applied together. Now ask the obvious question — *what if two parallel nodes both write to `messages`*? The default rule is "last write wins," but with parallel writes there is no "last." LangGraph refuses to guess and raises `InvalidUpdateError`.

A **reducer** is the answer: a function that tells LangGraph how to *combine* the existing value of a channel with a new value, instead of overwriting. It turns "two nodes wrote the same key" from an error into a well-defined merge.

Attaching a reducer with `Annotated`

The mechanism is Python's `Annotated`: the first argument is the type, the second is the reducer function.

```
from typing import Annotated, TypedDict
from operator import add

class State(TypedDict):
    count: int                # plain: last write wins
    items: Annotated[list, add] # reducer: concatenate lists
```

With the reducer in place, if one node returns `{"items": ["a"]}` and a parallel node returns `{"items": ["b"]}`, the final `items` is `["a", "b"]` — both survive. Without it, that same parallel write is an `InvalidUpdateError`.

```
def reducer(current: list, update: list) -> list:
    return current + update          # this is what operator.add does for lists
```

A reducer is just a pure function `(current_value, new_value) -> merged_value`. The intuition from everyday life: it's the rule for combining a shared shopping list when two people add to it — "merge everything," "keep only unique," or "take the latest." You pick the rule per channel.

The most important reducer: `add_messages`

Conversational agents accumulate a list of messages, and that list has special needs, so LangGraph ships a purpose-built reducer:

```
from typing import Annotated
from typing_extensions import TypedDict
from langchain_core.messages import AnyMessage
from langgraph.graph.message import add_messages
```

```
class State(TypedDict):
    messages: Annotated[list[AnyMessage], add_messages]
```

`add_messages` does more than append. Its superpowers — and the edge cases they quietly handle:

- **It appends new messages** to the list (the obvious part).
- **It updates by ID.** If a new message has the same `id` as an existing one, it *replaces* rather than duplicates. This is how you edit or stream-update a message in place — critical for token streaming and for correcting an earlier message.
- **It coerces formats.** Tuples like `("user", "hi")` and dicts get converted into proper `Message` objects.

***Edge case — duplicate messages from a missing-then-present ID.** If you construct messages without stable IDs and re-run a node, you can get duplicates because `add_messages` can't tell they're "the same" message. When you need replace-not-append semantics, give messages explicit, stable IDs. When you want a clean slate, `RemoveMessage` (a sentinel) lets you delete from the list through the same reducer.*

Because this pattern is so common, LangGraph provides a shortcut:

```
from langgraph.graph import MessagesState

class MyState(MessagesState):    # messages channel + add_messages, pre-wired
    summary: str                 # add your own plain fields
    user_id: str
```

`MessagesState` is a TypedDict with the `messages: Annotated[list, add_messages]` channel already declared. Subclass it for any chat-centric graph and add whatever extra channels you need.

When do you actually need a reducer? (intuition)

The decision rule:

- **Sequential writes, last-write-wins is fine** → no reducer. Most scalar fields (`status`, `current_step`, `answer`) are like this.
- **Multiple nodes accumulate into one channel** → you need a reducer. Message lists, collected results from a fan-out, running logs.
- **Parallel branches write the same key** → you *must* have a reducer, or it's an `InvalidUpdateError`.

A clean heuristic: **if a channel is a growing collection, give it an appending reducer; if it's a single current value, leave it plain.**

The subtle edge cases of reducers

Edge case — the reducer runs on every write, including from a single node. A reducer isn't only for parallelism. `{"items": ["x"]}` returned from a lone node still goes through the reducer, appending `["x"]` to whatever's there. This is why a node returning `{"messages": [new_msg]}` adds to the conversation rather than replacing it — exactly what you want, but only obvious once you see the reducer always fires.

Edge case — `None` and initial values. Your reducer must handle the channel's starting value. `operator.add` on a list works because the channel defaults to `[]`. But a custom reducer that assumes a non-empty current value will crash on the first write. Write reducers defensively: handle the empty/ `None` case explicitly.

Edge case — non-commutative reducers under parallelism. When two parallel nodes write the same channel, the order in which their updates are reduced is not guaranteed to match your graph's visual layout. If your reducer is `add` on lists, order may vary run to run. If correct results depend on ordering, either sort deterministically in the reducer or don't fan out into the same channel — collect into per-branch keys and merge in a downstream node where you control the order.

5. Nodes, Edges & Control Flow

Nodes: where work happens

A node is just a function (or async function) that takes the state and returns a partial update. That's the entire contract.

```
def my_node(state: State) -> dict:
    result = do_something(state["input"])
    return {"output": result}

builder.add_node("my_node", my_node)
```

Behind the scenes LangGraph wraps your function so it gains batching, async support, retries, tracing, and streaming — but you write plain Python. A node can do anything: call an LLM, hit an API, run a computation, transform state.

The `Runtime` object: context without globals

Nodes often need things that aren't part of the *data* state — a user ID, a database handle, the current `thread_id`, configuration. The modern way to get them is the `Runtime`, a second parameter LangGraph injects on request:

```
from dataclasses import dataclass
from langgraph.runtime import Runtime

@dataclass
class Context:
    user_id: str

def node(state: State, runtime: Runtime[Context]) -> dict:
    user = runtime.context.user_id           # static, per-run context
    thread = runtime.execution_info.thread_id # runtime execution info
    store = runtime.store                     # the long-term memory store
    return {"greeting": f"Hello, {user}!"}
```

The intuition: `state` is the data that flows and changes; `Runtime` is the ambient context for this run. Keep request-scoped configuration (user, tenant, feature flags) in context, not stuffed into state where it'd be checkpointed and reduced. You pass context at invoke time: `graph.invoke(input, context=Context(user_id="abc"))`.

Edges: the three kinds

Plain edges are unconditional: when source finishes, go to target.

```
builder.add_edge("a", "b")           # always a -> b
builder.add_edge(START, "a")         # entry point
```

```
builder.add_edge("b", END) # exit point
```

`START` and `END` are sentinel nodes: `START` is where input enters, `END` is where the graph stops. (The older `set_entry_point("a")` / `set_finish_point("b")` are equivalent to edges from `START` / to `END`.)

Conditional edges route dynamically based on a function you supply:

```
def route(state: State) -> str:
    if state["needs_tools"]:
        return "tools"
    return END

builder.add_conditional_edges("model", route)
```

The routing function receives the state and returns the *name(s)* of the next node(s). This is how an agent decides whether to loop back to tools or finish. You can also pass a mapping to make the wiring explicit and the graph diagram readable:

```
builder.add_conditional_edges("model", route, {"tools": "tools", END: END})
```

Entry/parallel fan-out. Adding multiple edges out of one node (or out of `START`) makes those targets run **in parallel** in the next super-step:

```
builder.add_edge(START, "researcher_a")
builder.add_edge(START, "researcher_b") # a and b run concurrently
```

This is the moment reducers become mandatory: if `researcher_a` and `researcher_b` both write the same channel, that channel needs a reducer.

Cycles: the feature that makes agents possible

Unlike a DAG, a LangGraph graph may contain cycles, and this is the whole point. The agent loop is `model → tools → model → tools → ...` until the model decides to stop. You express it with an ordinary edge from `tools` back to `model`:

```
builder.add_edge("tools", "model") # the cycle
```

The graph keeps looping until a conditional edge routes to `END`. Which raises the question every cyclic system must answer: *how does it terminate?*

Termination and the recursion limit (edge-case preview)

A cyclic graph that never routes to `END` would run forever. LangGraph's seatbelt is the **recursion limit** — the maximum number of super-steps in a single invocation, default **25**. Exceed it and you get a `GraphRecursionError`.

```
graph.invoke(input, {"recursion_limit": 50}) # raise the seatbelt if you truly need to
```

Edge case — the recursion limit counts super-steps, not loops. A single pass through `model → tools` is two super-steps. So the default 25 allows roughly a dozen tool-using turns, not 25. If your agent legitimately needs deep reasoning, raise the limit — but first ask whether it's looping productively or stuck. An agent that hits the limit is usually oscillating (calling the same tool with the same args, or two nodes ping-ponging), and the fix is a better stopping condition, not a bigger number. We devote Chapter 24 to designing termination properly.

Putting it together: the canonical agent graph by hand

Here is the full ReAct skeleton as a custom graph — the thing `create_agent` builds for you, written out so you can see every wire:

```
from langgraph.graph import StateGraph, MessagesState, START, END
from langgraph.prebuilt import ToolNode # still importable; ToolNode is low-level
from langchain.chat_models import init_chat_model

model = init_chat_model("anthropic:claude-sonnet-4-5").bind_tools(tools)

def call_model(state: MessagesState) -> dict:
    return {"messages": [model.invoke(state["messages"])]}

def should_continue(state: MessagesState) -> str:
    last = state["messages"][-1]
    return "tools" if last.tool_calls else END

builder = StateGraph(MessagesState)
builder.add_node("model", call_model)
builder.add_node("tools", ToolNode(tools))
builder.add_edge(START, "model")
builder.add_conditional_edges("model", should_continue)
builder.add_edge("tools", "model") # loop
graph = builder.compile()
```

Read that twice. The model node appends the model's reply (note `{"messages": [...]}` — the `add_messages` reducer appends it). The router checks whether the last message has tool calls. If yes, the tool node runs them and loops back; if no, we're done. This nine-line graph is a complete, correct agent. Everything in Part III is either a convenience on top of this or a way to control it more finely.

Part II — The Core Machinery

The features that make LangGraph more than a graph library: dynamic routing with `Command`, parallel fan-out with `Send`, durable persistence, time travel, human-in-the-loop, streaming, and memory. This is where the "durable state machine" promise gets cashed out.

6. Command: Update State and Route in One Move

The intuition

A conditional edge answers "where do I go next?" A node's return value answers "what changed in the state?" **Command** lets a node answer both at once — update the state *and* declare the next node, from inside the node itself.

```
from langgraph.types import Command
from typing import Literal

def my_node(state: State) -> Command[Literal["node_b", "node_c"]]:
    return Command(
        update={"foo": "bar"},      # state update
        goto="node_b",             # routing decision
    )
```

The `Literal[...]` type hint isn't decoration — it tells LangGraph (and the graph visualizer) which nodes this one can route to, so the compiled graph still has a complete picture of possible edges.

Command vs. conditional edges: the "ways" decision

You now have two ways to do dynamic routing. When does each win?

- **Conditional edges** keep routing logic *separate* from the node's work. The node computes; a dedicated routing function decides. This is cleaner when routing is a pure function of the resulting state, and it keeps the graph topology explicit and easy to read.
- **Command** fuses the decision into the node. This wins when **the routing decision and the state update are computed together** — the node already did the work that determines where to go, and splitting it into a separate router would mean recomputing or stashing intermediate values. It's also the natural tool for **multi-agent handoffs**, where an agent node decides "I'm handing off to the billing agent" as part of its own logic.

The intuition: reach for conditional edges by default (topology stays visible); reach for **Command** when the route is a byproduct of the node's own computation, or when you're doing agent-to-agent handoff.

Navigating to a parent graph

A **Command** can route not just within the current graph but **up to the parent graph** — essential for subgraphs and multi-agent systems where a sub-agent needs to hand control back:

```
return Command(
    goto="other_agent",
    update={"messages": [result]},
```

```
graph=Command.PARENT,      # route in the parent graph, not this subgraph
)
```

This is how a subgraph-as-agent says "I'm done, send control to a sibling agent in the parent."
Without `Command.PARENT`, `goto` is interpreted within the current (sub)graph's namespace.

Edge case — `Command` routing and reducers still apply. The `update` in a `Command` goes through the same channel reducers as a normal return. If you `Command(update={"messages": [m]})`, the `add_messages` reducer appends `m` — it doesn't replace the list. People sometimes expect `Command` updates to be "special" and bypass reducers. They don't.

Edge case — a node returning `Command` shouldn't also rely on outgoing static edges for routing. If a node returns a `Command(goto=...)`, that's the routing decision. Mixing it with `add_conditional_edges` on the same node muddies the model. Pick one routing mechanism per node.

7. The Send API & Map-Reduce

The problem: dynamic, data-dependent parallelism

Conditional edges and parallel fan-out both assume you know the branches at *design* time. But what about "fan out to one branch *per item* in a list whose length you won't know until runtime"? Generate N subtasks, process each in parallel, aggregate — the classic **map-reduce**, where N is dynamic.

Two specific challenges make this hard: the number of parallel branches is unknown when you draw the graph, and **each branch needs a different input** (its own item), not a copy of the shared state. The **Send API** solves both.

How Send works

From a conditional edge, instead of returning node *names*, you return a list of **Send** objects. Each **Send** targets a node **and carries a custom state** for that invocation:

```
from langgraph.types import Send

def continue_to_workers(state: OverallState):
    # one Send per item - N decided at runtime
    return [Send("process_item", {"item": item}) for item in state["items"]]

builder.add_conditional_edges("dispatcher", continue_to_workers)
```

Each `Send("process_item", {"item": item})` spins up an independent invocation of `process_item`, fed its own mini-state `{"item": item}`. The runtime runs them in parallel (subject to concurrency limits). Their outputs flow back into the main graph through a **reduced channel** — which is why the aggregation target needs a reducer:

```
class OverallState(TypedDict):
    items: list
    results: Annotated[list, add] # workers append here; reducer merges

def process_item(state: dict) -> dict:
    return {"results": [expensive(state["item"])]}
```

The intuition

Send is the bridge between "the graph's shared state" and "a per-branch private state." The mental model: the dispatcher *scatters* tailored work items; the workers each do one; a reduced channel *gathers* the results. The key insight that distinguishes **Send** from ordinary fan-out is that **the sent state can differ from — and need not be a subset of — the main state**. You're not splitting the state; you're injecting fresh, branch-specific inputs.

Edge cases that bite

Edge case — the aggregation channel MUST have a reducer. N workers writing `results` in parallel without a reducer is a guaranteed `InvalidUpdateError`. This is the single most common `Send` mistake. The reducer (`operator.add` for a list) is what makes parallel aggregation legal.

Edge case — ordering is not guaranteed. Workers finish in nondeterministic order, so `results` is in completion order, not input order. If you need to correlate outputs with inputs, include an index or key in each `Send` payload and sort downstream: `Send("worker", {"idx": i, "item": x})`, then reorder by `idx` after the gather.

Edge case — partial failure in a fan-out. If one of N workers raises, it can fail the whole super-step. For resilient fan-out, either make each worker catch its own errors and return a sentinel (`{"results": [{"error": ...}]}`) so the batch completes, or use the 1.2 node-level `error_handler` (Chapter 22) to convert a worker failure into a compensating result instead of a crash.

Edge case — unbounded fan-out. `Send` over a list of 10,000 items launches 10,000 concurrent invocations and 10,000 model calls if each worker calls an LLM. That's a cost and rate-limit grenade. Bound it: batch items per worker, or gate concurrency. Parallelism is a tool, not a default.

8. Persistence & Checkpointers

The intuition: a save point after every step

This is the feature that defines LangGraph. A **checkpointer** saves a snapshot of the graph's state after every super-step. That snapshot — the value of every channel, which node ran, any pending interrupts — is written to a durable store keyed by a `thread_id`. The consequences are everything LangGraph is known for:

- **Memory across sessions.** Resume a conversation days later; the message history is reconstructed from the last checkpoint.
- **Fault tolerance.** A crash mid-workflow doesn't lose work; replay from the last checkpoint instead of from zero.
- **Human-in-the-loop.** Pause indefinitely waiting for a human; the frozen state lives in the store, no process needs to stay alive.
- **Time travel.** Inspect, fork, or rewind to any past checkpoint.

The video-game metaphor is exact: a checkpoint is a save point recording your *entire* position — every variable, the message history, where execution stopped — so you can reload and continue.

Threads: the unit of persistence

A **thread** is one conversation or task. Every run is tagged with a `thread_id`, and the checkpointer keeps threads separate. The `thread_id` is, precisely, **your cursor into saved state**:

```
config = {"configurable": {"thread_id": "user-42-session-7"}}
graph.invoke({"messages": [("user", "Hi")]}], config)
# ...later, same thread_id resumes the same conversation:
graph.invoke({"messages": [("user", "What did I just say?")]}], config)
```

Reuse a `thread_id` → resume that thread's checkpoint. Use a new one → start fresh with empty state. This one detail is the source of a whole class of bugs (Chapter 21): the "agent forgot everything" report is almost always a `thread_id` that changed between turns.

Choosing a checkpointer (the "ways")

```
# Development / tests – lives in RAM, lost on process exit
from langgraph.checkpoint.memory import InMemorySaver
checkpointer = InMemorySaver() # MemorySaver is an alias

# Local persistence – a file on disk
from langgraph.checkpoint.sqlite import SqliteSaver

# Production – Postgres-backed, concurrent, durable
from langgraph.checkpoint.postgres import PostgresSaver
```

```
graph = builder.compile(checkpointer=checkpointer)
```

The beautiful part: **switching from dev to prod is one line**. Your graph logic doesn't change; only the checkpointer instance does. LangGraph abstracts the storage backend entirely.

Edge case — `InMemorySaver` is not restart-durable. This is the most consequential persistence gotcha. `InMemorySaver` lives in the process's RAM. Restart the process — a deploy, a crash, a Kubernetes pod eviction — and every frozen thread is gone. It's perfect for demos and tests and wrong for anything where state must outlive the process, especially a human-in-the-loop pause that might last hours. The instant you go to production, or the instant a human could take longer than your uptime, switch to `SqliteSaver`, Postgres, or Redis. Treat `InMemorySaver` in a deployed service as a latent outage.

What a checkpoint costs, and `DeltaChannel`

Naively, a checkpoint serializes the *entire* state at every step. For a long chat thread, that means re-serializing the whole growing message list on every single turn — checkpoint size grows with conversation length, and write cost grows with it.

LangGraph 1.2 introduced `DeltaChannel` (beta) precisely for this: instead of re-serializing the full accumulated value each step, it stores only the *incremental delta* written that step. For long-running threads, this keeps checkpoint sizes flat as the thread grows. The intuition: append-only channels (like message history) don't need to rewrite their entire contents every checkpoint — they only need to record what was added.

Edge case — don't put large binary blobs in state. A common, expensive mistake: stuffing PDFs, high-res images, or massive JSON datasets directly into a state channel. Every checkpoint then serializes that blob, ballooning storage and slowing writes. Store the artifact in a blob store (S3, a filesystem) and keep only a reference (a URL or key) in state. State is for the data that drives control flow, not for payloads.

The read-execute-write cycle

Mechanically, every invocation follows the same loop, which is worth knowing for debugging:

1. **Retrieve** — given the `thread_id`, load the latest checkpoint.
2. **Initialize** — hydrate the state (channels, message history, flags) into memory.
3. **Execute** — run the next super-step.
4. **Write** — persist the new checkpoint.

When something "doesn't remember," the bug is almost always in step 1–2: wrong `thread_id`, or a checkpointer that lost the data (in-memory across a restart).

9. Time Travel: `get_state`, `update_state` & Replaying

The intuition

Because every super-step is checkpointed, the history of a thread is a *sequence* of states you can walk. **Time travel** means inspecting any past checkpoint, editing one, or resuming execution from one. It turns "what was my agent thinking three steps ago?" into a query, and "what if it had decided differently?" into a fork.

Inspecting the current and past states

```
config = {"configurable": {"thread_id": "abc"}}

# Current state snapshot
snapshot = graph.get_state(config)
print(snapshot.values)      # channel values right now
print(snapshot.next)        # which node(s) run next (empty if done)
print(snapshot.config)      # includes the checkpoint_id of this snapshot

# Full history, newest first
for state in graph.get_state_history(config):
    print(state.config["configurable"]["checkpoint_id"], state.next)
```

`snapshot.next` is especially useful: if it's non-empty, the graph is paused (often at an interrupt) and those are the nodes that will run on resume. If it's empty, the run completed.

Editing state mid-run with `update_state`

You can write into a thread's state from outside the graph — the manual counterpart to a node's return:

```
graph.update_state(config, {"messages": [HumanMessage("Actually, use EUR not USD")]})
```

This goes through the channel reducers (so `add_messages` appends), produces a *new* checkpoint, and lets you correct or inject data before resuming. It's the foundation of one human-in-the-loop style: pause, let a human call `update_state` to fix the agent's plan, then resume.

Edge case — `as_node` and where the edit "came from." `update_state` accepts an `as_node=` argument that tells the graph to treat the update as if a particular node produced it, which determines what runs next. Omit it and `LangGraph` infers the position; get it wrong and resume can route from an unexpected place. When manually steering a paused graph, be deliberate about `as_node`.

Forking: exploring an alternate timeline

Pass a specific `checkpoint_id` and you resume from *that* point, not the latest — creating a branch:

```
fork_config = {"configurable": {"thread_id": "abc", "checkpoint_id": "<id-from-history>"}}
graph.update_state(fork_config, {"some_key": "different_value"})
graph.invoke(None, fork_config)    # runs forward from the fork with the changed value
```

This is invaluable for debugging ("rewind to before the bad tool call, change the input, see if it recovers") and for building "what-if" exploration into an app. Each fork is its own line of checkpoints under the same thread.

Replaying vs. resuming (a distinction that matters)

- **Resuming** continues a *paused* graph forward (e.g., after an interrupt).
- **Replaying** re-executes from a past checkpoint. Here's the subtlety: when you replay, **steps that already completed before the chosen checkpoint are not re-run** — LangGraph knows their results from the checkpoint and replays them from memory; only steps *after* the checkpoint actually execute. This is what makes fault recovery cheap: you don't redo the expensive work that already succeeded.

***Edge case — replaying re-executes nodes after the checkpoint, including their side effects.** Replay is free for completed steps but live for steps after the fork point. If those nodes have side effects (sending email, charging a card), replaying re-fires them. Same lesson as interrupts: keep side effects idempotent or guarded if your graph can be replayed.*

10. Human-in-the-Loop: interrupt() Deeply

This is the chapter where the durable-state-machine model pays off most, and where the most surprising edge case in all of LangGraph lives. Read it slowly.

Why human-in-the-loop is a persistence feature

As agents gain real power — writing to databases, sending email, spending money, committing code — letting them act unsupervised becomes dangerous. You want to **pause before a consequential action, get a human's approval, and resume**. That pause might last seconds or days. The only way to pause for days without a process babysitting the run is to *persist the frozen state and walk away* — which is exactly what checkpointing provides. **Human-in-the-loop is checkpointing with a human in the gap**. No checkpointer, no interrupts. Period.

The `interrupt()` function

```
from langgraph.types import interrupt, Command

def approval_node(state: State):
    decision = interrupt({
        "question": "Approve this $5,000 refund?",
        "details": state["refund"],
    })
    if decision == "approve":
        return {"status": "approved"}
    return {"status": "rejected"}
```

When `interrupt()` is called, the runtime saves the state and **stops**, returning control to your application with the interrupt payload visible. The graph is now frozen in the checkpointer. To resume, you invoke again with a `Command(resume=...)`, and that resume value becomes the **return value of `interrupt()`**:

```
config = {"configurable": {"thread_id": "t1"}}
graph.invoke(initial_input, config) # runs until interrupt, then returns

# inspect the pending interrupt
state = graph.get_state(config)
# ... show it to a human, collect their decision ...

graph.invoke(Command(resume="approve"), config) # resume; interrupt() returns "approve"
```

Requirements, non-negotiable: a **checkpointer** (to persist the frozen state) and a `thread_id` (so resume finds the right frozen state). Same `thread_id` for the initial call and the resume — that's how the checkpointer knows which frozen state to restore.

THE edge case: the node re-runs from the top on resume

This is the single most important thing to understand about interrupts, and it surprises nearly everyone.

When you resume, LangGraph does not continue from the line after `interrupt()`. It re-executes the entire node from the beginning. The `interrupt()` call is reached again, but this time — instead of pausing — it immediately returns the resume value and execution continues past it.

Why? Because the checkpoint granularity is the super-step, not the line. The runtime can only resume at a node boundary, so resuming means re-running the node, with `interrupt()` acting as a "replay the answer this time" marker.

The consequence is stark:

```
def buggy_node(state: State):
    charge_credit_card(state["amount"])    # ⚠️ SIDE EFFECT before interrupt
    ok = interrupt("Confirm the charge?")  # node re-runs from top on resume...
    return {"done": ok}                   # ...so charge_credit_card runs AGAIN
```

On resume, `charge_credit_card` runs a **second time** — the customer is charged twice. The fix is a rule you must internalize:

The idempotency rule for interrupts: pure computation before `interrupt()`, side effects after. Anything that runs before the interrupt will run again on resume. Put reads and computation before; put writes, charges, sends, and irreversible actions after the `interrupt()` returns.

```
def correct_node(state: State):
    summary = compute_summary(state)      # pure, safe to repeat
    ok = interrupt({"confirm": summary})   # pause
    if ok == "approve":
        charge_credit_card(state["amount"]) # side effect AFTER resume – runs once
    return {"done": True}
    return {"done": False}
```

THE second edge case: never wrap `interrupt()` in `try/except`

`interrupt()` works by **raising a special exception** (`GraphInterrupt`) that the runtime catches at the top of the execution loop to trigger the pause. If you wrap the call in a `try/except` that catches broad exceptions, **you swallow the interrupt** and the pause never happens:

```
try:
    decision = interrupt("Approve?")    # ⚠️ the GraphInterrupt is caught below
except Exception:
    decision = "default"                 # interrupt silently broken
```

Edge case — bare `except Exception` around `interrupt()` defeats it. Keep `interrupt()` out of broad try/except blocks. If you must have error handling nearby, catch specific exceptions, never the base class, and let `GraphInterrupt` propagate.

Static vs. dynamic interrupts (the "ways")

There are two ways to pause, and they suit different needs.

Dynamic — `interrupt()` inside a node. The pause is conditional on runtime state ("only if the refund exceeds \$1,000"), and you control the payload. This is the modern, flexible default.

```
def review(state):
    if state["amount"] > 1000:
        decision = interrupt({"amount": state["amount"]})
        ...
```

Static — `interrupt_before` / `interrupt_after` at compile time. You declare, when compiling, that the graph should pause before or after specific nodes — no code inside the node needed.

```
graph = builder.compile(checkpointer=cp, interrupt_before=["sensitive_tool"])
```

The intuition: use **static** interrupts for a blanket "always pause before this node" policy (great for debugging — pause before every step and inspect) or simple approval gates. Use **dynamic** `interrupt()` when the pause is conditional, needs a rich payload, or needs to collect structured input. Dynamic is more powerful; static is simpler and requires no node changes.

Resuming multiple parallel interrupts

If you fan out (Chapter 7) into several nodes that each call `interrupt()`, all of them pause in the same super-step. You then have multiple pending interrupts at once. Resuming requires mapping **each interrupt's ID to its resume value**:

```
state = graph.get_state(config)
# state.tasks / the __interrupt__ payload expose each Interrupt with an .id
graph.invoke(Command(resume={interrupt_id_a: "approve", interrupt_id_b: "reject"}), config)
```

Edge case — a single resume value for multiple interrupts pairs wrong. With parallel interrupts, `Command(resume="approve")` (a bare value) is ambiguous — which branch gets it? Map by interrupt ID so each human answer lands on the correct paused branch.

The "never resumes" case

Humans are not reliable responders. Someone gets pulled into a meeting and your thread sits frozen forever. Design for it:

Edge case — design a TTL for abandoned interrupts. A frozen thread consumes storage and represents an undelivered outcome. Build an expiry mechanism: periodically query the checkpoint for threads whose `updated_at` exceeds your TTL, then auto-reject, escalate, or re-notify. Never assume the human comes back.

The legacy pattern: `NodeInterrupt`

You'll see older code raise `NodeInterrupt("reason")` from a node, then resume by calling `update_state` to inject the human's response and re-invoking *without* a `Command`. This predates the `interrupt()` + `Command(resume=...)` model. It still works, but the modern function-based approach is cleaner and is what new code should use. Recognize `NodeInterrupt` so you can read old codebases; reach for `interrupt()` when writing new ones.

How agents get human-in-the-loop for free

In the high-level world, you don't wire any of this by hand — you add `HumanInTheLoopMiddleware` (Chapter 15) to a `create_agent`, and it automatically interrupts before configured tool calls for approval. Under the hood it's exactly the `interrupt()` / `Command(resume=...)` machinery above, packaged so a sensitive tool ("process_refund") pauses for sign-off with no graph surgery. Knowing the primitive lets you understand and debug the middleware.

11. Streaming

Why streaming is not optional

An agent that thinks for thirty seconds and then dumps a wall of text feels broken, even if it's correct. Streaming — surfacing tokens, tool calls, and intermediate state *as they happen* — is what makes agents feel responsive. LangGraph has rich, first-class streaming, and the key to using it is knowing **which mode emits what**.

The stream modes (a "ways" decision you'll make constantly)

`graph.stream(input, config, stream_mode=...)` (and async `astream`) accept several modes:

- **"values"** — emits the **full state** after each super-step. Use when you want the complete, current picture each tick (e.g., re-render an entire UI from state). Heavier, but simplest to consume.
- **"updates"** — emits **only the changes** each node made, as `{node_name: {changed_key: value}}`. Use when you want to react to *what each node did* without diffing full states. Lighter and more granular.
- **"messages"** — emits **LLM tokens** as they're generated, along with metadata, for token-by-token streaming to a UI. This is the "ChatGPT typing" effect.
- **"custom"** — emits whatever *you* push from inside a node via a writer (see below). For surfacing progress from non-LLM work ("fetched 3 of 10 documents").
- **"debug"** — emits maximally verbose events (every node start/finish, every state write). For development and tracing.

```
for chunk in graph.stream(input, config, stream_mode="updates"):
    print(chunk)      # {'model': {'messages': [AIMessage(...)]}}, then {'tools': {...}}, ...
```

You can request **multiple modes at once** by passing a list, and each emitted event is tagged with its mode:

```
for mode, chunk in graph.stream(input, config, stream_mode=["updates", "messages"]):
    if mode == "messages":
        token, meta = chunk
        ...
```

Streaming custom progress from inside a node

For long-running, non-LLM work, push your own updates with the stream writer from the runtime:

```
from langgraph.config import get_stream_writer

def long_node(state):
    writer = get_stream_writer()
    for i, doc in enumerate(documents):
```

```

    process(doc)
    writer({"progress": f"{i+1}/{len(documents)} documents"}) # shows up in
    stream_mode="custom"
    return {"done": True}

```

The intuition: `"custom"` is your escape hatch to stream *anything* meaningful to the user that isn't a token or a state change — progress bars, status text, partial results.

astream_events and the v3 API

`astream_events` is the most granular stream: a flat sequence of typed events (model start, model stream, tool start, tool end, chain events, and so on) across the entire graph, including nested subgraphs. It's what you use to build a fully custom streaming UI that reacts to specific lifecycle moments.

LangGraph 1.2 / LangChain introduced `version="v3"` for `astream_events` — a content-block-centric streaming model with typed, per-channel projections, aligning streamed events with the standard content-blocks representation that 1.0 made canonical:

```

async for event in graph.astream_events(input, config, version="v3"):
    if event["event"] == "on_chat_model_stream":
        ... # a token chunk, now as standardized content blocks

```

Edge case — `stream_mode="messages"` streams every LLM call in the graph. If your graph has a model node and a summarization node that both call an LLM, `"messages"` streams tokens from both. Use the metadata attached to each token (which carries the node name / tags) to filter to the model whose tokens you actually want to show. Streaming "leaks" from helper LLMs is a common UI bug.

Edge case — streaming and interrupts interact. When a graph interrupts mid-stream, the stream ends at the interrupt; the pending interrupt appears in the final emitted chunk (and via `get_state`). Your consumer must treat "stream ended with a pending interrupt" differently from "stream ended because the graph finished." Check `snapshot.next` / the `__interrupt__` payload to tell them apart.

12. Memory: Short-Term, Long-Term & the Store

Two kinds of memory, two mechanisms

"Memory" is overloaded. LangGraph cleanly separates two notions, and conflating them is a common design error:

- **Short-term (thread-scoped) memory** is the state of *one* conversation — the message history and channels for a single `thread_id`. It's provided by the **checkpointer**. When you resume a thread, the short-term memory comes back. It does *not* cross threads: user A's session knows nothing of user B's, and even the same user's two sessions are separate threads.
- **Long-term (cross-thread) memory** is knowledge that should persist *across* conversations and users — preferences, facts learned about a user, a knowledge base. It's provided by the **Store**, a separate key-value system with namespacing and optional semantic search.

The intuition: **checkpointer = "what happened in this conversation"; store = "what I know that outlives any conversation."** A chatbot uses the checkpointer to remember the current chat and the store to remember "this user prefers metric units" forever.

The Store API

```
from langgraph.store.memory import InMemoryStore  # dev; use a DB-backed store in prod

store = InMemoryStore()
graph = builder.compile(checkpointer=checkpointer, store=store)

# Inside a node, reach the store via the runtime:
def remember(state, runtime):
    store = runtime.store
    namespace = ("users", runtime.context.user_id, "preferences")
    store.put(namespace, "units", {"value": "metric"})      # write
    pref = store.get(namespace, "units")                    # read
    hits = store.search(namespace, query="measurement system") # semantic search (if configured)
    return {...}
```

Namespaces are tuples that organize memory hierarchically — `("users", user_id, "preferences")` — so you can scope, list, and search within a slice. With an embeddings configuration, `search` does **semantic** retrieval, turning the store into a lightweight long-term memory / RAG substrate the agent can query for relevant facts.

How agents use long-term memory

In a `create_agent`, you pass a `store=` and access it from middleware via `request.runtime.store` to inject relevant memories into the prompt before the model call, or to write new memories after. The pattern: **before the model runs, retrieve relevant long-term memories and add them to**

context; after, extract and persist anything worth remembering. This is the backbone of personalized agents and is exactly what middleware hooks (Chapter 14) are designed to do.

Edge cases and discipline

Edge case — don't confuse the store with the checkpointer. Putting per-conversation scratch in the store pollutes long-term memory with ephemeral junk; putting durable user facts only in thread state means they vanish when the thread is archived or the user starts a new session. Match the lifetime of the data to the mechanism.

Edge case — `InMemoryStore` forgets on restart, just like `InMemorySaver`. For real long-term memory, back the store with a database. An in-memory store that loses your users' learned preferences on every deploy defeats the purpose.

Edge case — long-term memory grows unbounded and can poison context. If you accumulate memories forever and inject them all, you'll blow the context window and feed the model stale or contradictory facts. Curate: deduplicate, summarize, expire, and retrieve selectively (top-k by relevance) rather than dumping everything. Memory you never prune becomes a liability, not an asset.

Edge case — `WithValue`-style writes need consistency under concurrency. Two threads for the same user writing the same namespace key can race (last-write-wins). For counters or sets that multiple sessions update, design idempotent or merge-friendly writes rather than blind overwrites.

Part III — Building Agents

The practical assembly. Six chapters on the concrete ways to build an agent: the prebuilt `create_agent`, the middleware that customizes it, tools, dropping to a custom graph, multi-agent systems, and subgraphs. This is where intuition becomes shippable software.

13. The Prebuilt Agent: `create_agent`

Start here, always

`create_agent` (in `langchain.agents`, the 1.0 successor to `langgraph.prebuilt.create_react_agent`) builds the full ReAct agent — the model/tools loop from Chapter 2 — in one call, with a clean path to customize via middleware. The mastery mindset is *not* "real engineers build custom graphs." It's "use the highest-level tool that fits, and drop down only when you must." For most agents, `create_agent` fits.

```
from langchain.agents import create_agent

agent = create_agent(
    model="anthropic:claude-sonnet-4-5",
    tools=[search, calculator, send_email],
    system_prompt="You are a careful research assistant. Cite sources.",
)
result = agent.invoke({"messages": [("user", "Summarize today's AI news with sources.")]}))
print(result["messages"][-1].content)
```

It returns a **compiled LangGraph graph** (a `CompiledStateGraph`). That means everything from Part II applies unchanged: pass a `checkpointers` for memory, a `store` for long-term memory, `stream` it, interrupt it, compose it as a node in a bigger graph.

The parameters that matter

- `model` — a provider string (`"openai:gpt-5.5"` , `"anthropic:claude-sonnet-4-5"`) or a `BaseChatModel` instance. Note: when using middleware, the model must be a string or `BaseChatModel` (not a pre-bound runnable).
- `tools` — a list of tools (functions decorated with `@tool` , or `BaseTool` instances). An empty list yields a model-only agent with no tool loop — just a structured LLM call.
- `system_prompt` — a string (becomes a `SystemMessage`) or `None` . With middleware, prompts must be strings or `None` , not templated runnables.
- `response_format` — a schema (Pydantic model, `TypedDict` , or JSON schema) for **structured output**. The agent will return data matching the schema, handled inside the loop.
- `middleware` — the list of middleware that customize behavior (Chapter 14). This is the primary extension point.
- `checkpointers` / `store` — persistence and long-term memory, exactly as in Part II.
- `context_schema` — the schema for runtime context (the `Runtime` object's `.context`).
- `name` — names the compiled graph; used automatically when this agent becomes a subgraph node in a multi-agent system.

Structured output, the modern way

A 1.0 breaking change worth internalizing: **prompted JSON is gone**. You no longer ask the model to "respond in JSON" via the prompt. Instead, you give a *schema* and the agent enforces it:

```
from pydantic import BaseModel

class Ticket(BaseModel):
    category: str
    priority: int
    summary: str

agent = create_agent(model="...", tools=[...], response_format=Ticket)
result = agent.invoke({"messages": [("user", "My laptop won't turn on and I have a demo in an hour!")]}))
result["structured_response"] # -> Ticket(category="hardware", priority=1, summary="...")
```

Under the hood, the agent picks a **strategy**: `ProviderStrategy` (use the model provider's native structured-output API, when available) or `ToolStrategy` (emulate structured output via a tool call). It auto-selects based on model capabilities, or you can force one. The intuition: structured output is now a *typed contract*, validated, not a prompt-engineering hope.

When `create_agent` is enough — and when it isn't

`create_agent` covers: a single agent that calls tools in a loop, optionally with structured output, memory, human-in-the-loop (via middleware), summarization, retries, guardrails. That is a *lot* of real applications.

You outgrow it when you need to **shape the control flow itself**: multiple distinct nodes with deterministic routing between them, a non-agentic step wired into the middle, several cooperating sub-agents with custom handoff logic, or state that's richer than a message list plus a few fields. That's the signal to drop to a custom `StateGraph` (Chapter 17) — often while still using `create_agent` to build the individual agents *inside* that graph.

14. Middleware: The Modern Control Surface

The intuition

LangChain 1.0's headline feature is **middleware**: composable hooks that intercept the agent loop at well-defined points. The analogy the designers intend is web-server middleware — Express.js or Django — but for the agent's think/act loop. Instead of a pile of constructor flags (which led to "parameter explosion"), you compose small, single-purpose, testable middleware like Lego blocks.

Middleware exists to solve **context engineering**: precise control over what goes into the model at each step, and what happens to what comes out. The hard-won lesson behind it is that production agents fail less because the loop is wrong and more because the *context* flowing into the model is wrong — too much, too little, stale, or unsafe. Middleware is the surgical tool for that.

The lifecycle hooks

Middleware can implement any of these (each has a sync and async form):

- **before_agent** — runs once before the agent starts. Setup, loading memory, validating input.
- **before_model** — runs before *each* model call. The prime spot to inject context: retrieve memories, trim history, add a dynamic system prompt.
- **wrap_model_call** — wraps the model call with a handler you control. You can call the handler multiple times (**retries**), skip it (**short-circuit**), or modify the request/response. This is the most powerful hook.
- **after_model** — runs after each model call. Inspect or validate the model's output, enforce guardrails, post-process.
- **wrap_tool_call** — intercepts tool execution: retries, monitoring, modification, or replacing the result.
- **after_agent** — runs once after the agent finishes. Cleanup, persisting memories, final logging.

The request flows *down* through **before_model** hooks (first middleware in the list outermost) and the response flows *back up* through **after_model** hooks in reverse — a classic onion. Composition order is explicit: **the first middleware in the list is the outermost layer**.

```
before_model:  M1 -> M2 -> M3 -> [MODEL] -> M3 -> M2 -> M1  :after_model
```

Writing custom middleware

Two ways, mirroring the framework's general philosophy of "easy thing easy, hard thing possible."

Decorator form for quick, single-hook middleware:

```
from langchain.agents.middleware import before_model, after_model, dynamic_prompt
```

```

@dynamic_prompt
def add_user_context(request):
    user = request.runtime.context.user_id
    prefs = request.runtime.store.get(("users", user), "prefs")
    return f"You are helping user {user}. Their preferences: {prefs}."

@after_model
def block_pii(state, runtime):
    last = state["messages"][-1]
    if contains_ssn(last.content):
        return {"messages": [redact(last)]}
    return None # None = no change

```

Class form for richer middleware with multiple hooks and state:

```

from langchain.agents.middleware import AgentMiddleware

class BudgetMiddleware(AgentMiddleware):
    def __init__(self, max_calls: int):
        self.max_calls = max_calls

    def before_model(self, state, runtime):
        calls = state.get("model_calls", 0)
        if calls >= self.max_calls:
            # short-circuit: stop calling the model
            return {"messages": [AIMessage("I've hit my reasoning budget.")], "jump_to": "end"}
        return {"model_calls": calls + 1}

```

Note `jump_to` — middleware can redirect the agent's control flow (e.g., jump straight to the end), which is how short-circuiting guardrails work.

ModelRequest and ModelResponse

The objects middleware manipulate:

- **ModelRequest** — the about-to-be-sent request: the messages, the system prompt, the tools, the model, and `runtime`. Middleware reads and rewrites this to engineer context (swap the model based on complexity, trim messages, change the prompt).
- **ModelResponse** — what came back: the messages and any structured output. Middleware inspects and validates it.

The intuition: `before_model` / `wrap_model_call` is your chance to control **what the model sees**; `after_model` is your chance to control **what you do with what it said**. Almost every production concern — cost, safety, personalization, reliability — is one of those two.

Why middleware beats the old way

The anti-pattern it replaces: *before middleware, customizing an agent meant either subclassing internals or threading a dozen special-case parameters through the constructor. Behavior was tangled and untestable. Middleware makes each concern a separate, unit-testable object you can add incrementally. The recommended workflow is exactly that: **add one middleware, test it, then add the next.** Start simple; compose up.*

15. The Built-in Middleware Catalog

LangChain ships production-ready middleware for the concerns that come up again and again. Knowing this catalog means you rarely build from scratch — you compose. Reach for these by name:

Context management

- `SummarizationMiddleware` — watches the conversation and, as it approaches the token limit, automatically condenses older messages into a summary so long chats stay within budget. The standard answer to "my conversation got too long."
- `LLMToolSelectorMiddleware` — when an agent has many tools, uses an LLM to pre-select the relevant subset *before* the main model call, so the model isn't distracted by (and doesn't pay for) dozens of irrelevant tool schemas.

Safety and guardrails

- `HumanInTheLoopMiddleware` — pauses execution for human approval before configured (sensitive) tool calls. This is the packaged form of Chapter 10's `interrupt()` machinery — add it, list the tools that need sign-off, and the agent halts for approval on those, with no graph surgery. Requires a checkpoint.
- `PIIMiddleware` — detects personally identifiable information (emails, phone numbers, SSNs, API keys, custom patterns) and applies a strategy: `block`, `mask`, or `redact`. Add one per PII type. Makes outputs safe to log and display.
- `ModelCallLimitMiddleware` / `ToolCallLimitMiddleware` — hard caps on how many model calls or tool calls an agent may make, enforced mid-run. The reliable defense against runaway loops and cost blowouts (see Chapter 24).

Reliability

- `ModelFallbackMiddleware` — automatically falls back to alternative models on error. Primary model down or rate-limited? Transparently retry on a backup. Essential for production uptime.
- `ModelRetryMiddleware` — retries failed model calls with exponential backoff, with configurable `max_retries`, `backoff_factor`, and an exception filter (`retry_on`, or a predicate to retry only on, say, HTTP 429/503).
- `ToolRetryMiddleware` — retries failed *tool* calls with backoff. On exhaustion, choose `return_message` (hand the error back to the LLM to handle), `raise` (stop the agent), or a custom function that converts the exception into a `ToolMessage`.

Capability and structure

- `TodoListMiddleware` — gives the agent a `write_todos` tool and tracks task completion across a multi-step job, forcing it to plan before acting. A core ingredient of "deep agents" that work over long horizons.

- **ShellToolMiddleware** — registers a persistent shell tool with execution policies (**HostExecutionPolicy** or sandboxed **DockerExecutionPolicy**), startup commands, and output redaction rules. For agents that run commands.
- **Filesystem middleware** — provides **Glob** and **Grep** search over files, giving an agent structured filesystem access.
- **LLMToolEmulator** — emulates specified tools with an LLM instead of executing them — invaluable for testing an agent's behavior without hitting real APIs.
- **AnthropicPromptCachingMiddleware** (from `langchain_anthropic.middleware`) — enables provider prompt caching with a TTL, slashing cost on multi-turn conversations by reusing cached prefixes.

Composing them: a production agent

```
from langchain.agents import create_agent
from langchain.agents.middleware import (
    SummarizationMiddleware, HumanInTheLoopMiddleware,
    PIIMiddleware, ModelCallLimitMiddleware,
)
from langchain.checkpointers import MemorySaver  # or langgraph.checkpoint.memory

agent = create_agent(
    model="anthropic:claude-sonnet-4-5",
    tools=[search, process_refund, send_email],
    middleware=[
        PIIMiddleware("email", strategy="mask"),           # outermost: scrub first
        SummarizationMiddleware(),                         # keep context bounded
        ModelCallLimitMiddleware(max_calls=15),           # cap runaway loops
        HumanInTheLoopMiddleware(                         # approval gate
            interrupt_on={"process_refund": True, "send_email": True},
        ),
    ],
    checkpointer=MemorySaver(),  # required for HumanInTheLoop
)
```

The intuition for ordering: put **input-scrubbing** (PII) outermost so everything downstream sees clean data; put **budget/limit** guards where they'll catch loops; put **approval gates** close to the tools they protect. Order is behavior, so be deliberate.

Edge case — **HumanInTheLoopMiddleware** **without a checkpointer is a no-op-that-crashes**. Like raw interrupts, it needs persistence to freeze and resume. Forgetting the `checker=` is the most common setup error. And in production, that checkpointer must be durable (not **MemorySaver**) or an approval that takes longer than your process uptime is lost.

16. Tools & Tool Calling

What a tool is

A tool is a function the model can choose to call. You describe it (name, description, argument schema); the model, given the descriptions, decides when to call which tool and with what arguments; you execute it and return the result. The model's *only* power is to **request** a call — your code decides whether and how to run it. That separation is a safety boundary worth remembering.

```
from langchain_core.tools import tool

@tool
def get_weather(city: str) -> str:
    """Get the current weather for a city.""" # the docstring is the model's instructions
    return weather_api(city)
```

The docstring and type hints *are* the interface the model sees. Write them as if instructing the model — because you are. A vague description yields wrong or missing calls; a precise one ("city: the city name, e.g. 'Paris, France'") yields correct ones. Tool descriptions are prompt engineering.

Binding tools and the ToolNode

At the low level, you bind tools to a model (so it knows their schemas) and execute requested calls with a `ToolNode`:

```
from langgraph.prebuilt import ToolNode

model_with_tools = model.bind_tools([get_weather, search])
tool_node = ToolNode([get_weather, search])
```

`.bind_tools()` is provider-agnostic — the same call works across OpenAI, Anthropic, and others, which is what makes graph code model-portable. `ToolNode` takes the model's tool-call requests from the last message, executes them (in parallel if there are several), and returns the results as `ToolMessage`s. In a `create_agent`, both are handled for you; you wire them by hand only in a custom graph.

Tools that drive control flow: returning Command

A tool can return a `Command` instead of a plain value, letting a tool call *update graph state* or *route* — the mechanism behind multi-agent handoffs:

```
from langgraph.types import Command

@tool
def transfer_to_billing(state) -> Command:
```

```
"""Hand off to the billing specialist."""  
return Command(goto="billing_agent", update={"messages": [...]}, graph=Command.PARENT)
```

The intuition: most tools return data the model reads next turn; *handoff* tools return a `Command` that moves control. This is how "the agent decided to escalate" becomes a concrete routing action.

Tool-calling edge cases

Edge case — parallel tool calls and state. A model can request several tool calls in one turn; `ToolNode` runs them concurrently. If those tools write to the same state channel, you need a reducer — same super-step rule as everywhere. Tools aren't exempt from the parallelism model.

Edge case — tool errors and `handle_tool_errors`. By default, an exception in a tool can surface as an error `ToolMessage` (so the model can react) or propagate, depending on configuration. `ToolNode(..., handle_tool_errors=...)` controls this. Decide deliberately: do you want the model to see the error and retry differently (often yes for transient failures), or do you want the run to fail loudly (yes for programmer errors)? Silent error-swallowing that returns a confusing message to the model causes agents to loop trying to "fix" something they can't see.

Edge case — the model hallucinates a tool or bad arguments. Models sometimes request a non-existent tool or pass arguments that violate the schema. Robust agents validate tool calls and return a corrective `ToolMessage` ("no such tool" / "argument X must be an integer") so the model can self-correct, rather than crashing. Schema validation on tool args (via Pydantic) catches most of this.

Edge case — long or large tool outputs blow the context window. A tool that returns a 50-page document dumps all of it into the next model call. Truncate, summarize, or store-and-reference large tool outputs (same discipline as large state). A tool's return is context the model must pay for.

Edge case — tools with side effects under retries/replays/interrupts. Everything from Chapters 9–10 applies: if a tool sends email or charges money, and the graph can replay or resume, that tool can fire twice. Make side-effecting tools idempotent (e.g., accept an idempotency key) or guard them behind a human-in-the-loop gate.

17. Custom Agents with StateGraph

When to drop down

You've seen the prebuilt agent and middleware. Now the other end of the spectrum: building the agent as an explicit `StateGraph`. The signal to do this is precise — you need **control over the control flow** that the prebuilt loop doesn't expose:

- A **deterministic step** wired between agentic ones ("after the agent drafts, always run a fact-check node, never optionally").
- A **bespoke routing decision** that isn't just "tools or done."
- **Richer state** than messages + a few fields — multiple accumulating channels, a plan, a scratchpad with structure.
- **Multiple sub-agents** with custom handoff logic (Chapter 18).
- A **human-in-the-loop pattern** more elaborate than "approve this tool."

The mindset: a custom graph isn't more "advanced" or more correct — it's the right tool when the *shape* of the computation is yours to design. And you can still build the individual agentic pieces with `create_agent` and drop them in as nodes.

Building ReAct by hand (the reference implementation)

We saw the skeleton in Chapter 5; here it is fleshed out with the patterns you'd actually add — a system prompt, a guard, and structured termination:

```
from langgraph.graph import StateGraph, MessagesState, START, END
from langgraph.prebuilt import ToolNode
from langchain.chat_models import init_chat_model
from langchain_core.messages import SystemMessage

tools = [search, calculator]
model = init_chat_model("anthropic:claude-sonnet-4-5").bind_tools(tools)
SYSTEM = SystemMessage("You are precise. Use tools for any factual claim.")

def call_model(state: MessagesState) -> dict:
    response = model.invoke([SYSTEM] + state["messages"])
    return {"messages": [response]}

def should_continue(state: MessagesState) -> str:
    last = state["messages"][-1]
    if not last.tool_calls:
        return END
    return "tools"

builder = StateGraph(MessagesState)
builder.add_node("model", call_model)
builder.add_node("tools", ToolNode(tools))
builder.add_edge(START, "model")
builder.add_conditional_edges("model", should_continue, {"tools": "tools", END: END})
```

```
builder.add_edge("tools", "model")
graph = builder.compile(checkpointer=checkpointer)
```

What customizing this buys you that the prebuilt doesn't: you can insert a node between `tools` and `model` (e.g., a node that filters or annotates tool results), swap the model per-turn based on state, add channels to `MessagesState`, or change the termination logic to something richer than "no tool calls."

Mixing deterministic and agentic — the LangGraph superpower

The pattern that justifies LangGraph's existence: a graph where some edges are fixed and some are LLM-chosen.

```
# Deterministic intake → agentic research → deterministic validation → agentic synthesis
builder.add_edge(START, "validate_input")      # fixed: always validate first
builder.add_edge("validate_input", "research") # fixed
builder.add_conditional_edges("research", needs_more_research) # agentic: loop or move on
builder.add_edge("research", "fact_check")     # fixed: always fact-check
builder.add_conditional_edges("fact_check", passed_check)    # agentic-ish: redo or finish
```

The intuition: **give the LLM authority only where judgment is genuinely needed, and keep everything else deterministic.** Each fixed edge is a place you've removed a chance for the agent to go wrong. This is why LangGraph agents can be made far more reliable than fully autonomous ones — you choose, per transition, who's in charge.

Edge case — forgetting the loop-back edge. Building a custom agent and omitting `add_edge("tools", "model")` produces an agent that calls a tool once and then halts, because nothing routes the tool result back to the model. The cycle is the agent; a missing back-edge is the most common custom-graph bug.

18. Multi-Agent Architectures

Why multiple agents

A single agent with fifty tools and a sprawling system prompt becomes confused, expensive, and unmaintainable. Splitting responsibilities across **specialized agents** — each with a focused toolset and prompt — improves reliability and modularity, at the cost of coordination complexity. The question is always *how they coordinate*, and there are a few canonical "ways," each a different answer to "who decides what runs next?"

The architectures (the "ways")

Supervisor. A central supervisor agent routes work to specialist agents and collects their results. Specialists don't talk to each other; everything goes through the hub. Easiest to reason about and control; the supervisor is a clean place to enforce policy. The default choice for most multi-agent systems.



Swarm / handoff. Agents hand control directly to one another via handoff tools (a tool that returns `Command(goto=other_agent)`). Control passes peer-to-peer; whichever agent is active "owns" the conversation until it hands off. More flexible, less centralized, harder to bound — control can wander.

Hierarchical. Supervisors of supervisors — teams of agents, each team with its own supervisor, coordinated by a top-level supervisor. For large systems that decompose into sub-domains. It's the supervisor pattern, nested.

Network. Any agent can call any other. Maximum flexibility, maximum chaos — hardest to debug and bound. Use only when the problem genuinely requires arbitrary agent-to-agent communication; usually a supervisor or hierarchy is better.

Implementing handoffs with Command

The mechanism under all of these is `Command`-based handoff, often wrapped in a tool:

```
from langgraph.types import Command

def make_handoff(target: str):
    @tool(name=f"transfer_to_{target}")
    def handoff(state) -> Command:
        f"""Transfer the conversation to the {target} agent."""
        return Command(
```

```

        goto=target,
        update={"messages": [ToolMessage(f"Transferred to {target}", ...)]},
        graph=Command.PARENT,      # route in the parent graph that holds all agents
    )
    return handoff

```

The supervisor (or each peer) is given handoff tools; calling one routes control to the named agent in the parent graph. `Command.PARENT` is essential when each agent is a subgraph — the handoff must navigate the *parent's* namespace where the sibling agents live.

Subgraphs as agents

The clean way to build multi-agent systems: **each agent is a compiled graph, added as a node in a parent graph**. Because `create_agent` returns a compiled graph, you can build each specialist with `create_agent` and compose them:

```

research_agent = create_agent(model=..., tools=[search], name="research")
coding_agent   = create_agent(model=..., tools=[run_code], name="coding")

parent = StateGraph(MessagesState)
parent.add_node("research", research_agent)    # an agent as a node
parent.add_node("coding", coding_agent)
parent.add_node("supervisor", supervisor_fn)
# ... edges / handoffs ...

```

The `name=` you gave each agent is used as its node identity — which is why `create_agent` exposes it.

State sharing: the central design decision

The hardest question in multi-agent design is **what state agents share**. Two regimes:

- **Shared message history.** All agents read and write one `messages` channel. Simple, and agents see each other's work — but the context grows fast and one agent's verbosity pollutes another's prompt.
- **Separate / scoped state.** Each agent has its own state; the parent passes only what's relevant across handoffs. More isolation and smaller contexts, but you must explicitly marshal information between agents.

Edge case — shared message history causes context explosion and cross-talk. If five agents all append to one `messages` list, every agent's context includes everyone's chatter, costs balloon, and a specialist gets distracted by irrelevant exchanges. Scope aggressively: pass summaries across handoffs, give sub-agents private message channels, and only surface to the parent what it needs.

Edge case — handoff loops. Agent A hands to B, B hands back to A, forever. With peer-to-peer handoffs there's no central budget. Enforce a handoff/step limit (the recursion limit catches it eventually, but design an explicit cap), and prefer a supervisor where one place can detect and break cycles.

Edge case — `Command.PARENT` vs local `goto`. Inside a subgraph-agent, a plain `goto="sibling"` looks for `sibling` within that subgraph and fails, because siblings live in the parent. Forgetting `graph=Command.PARENT` on handoffs is a frequent and confusing multi-agent bug.

19. Subgraphs

The intuition

A subgraph is a graph used as a node inside another graph. It's the unit of **composition and reuse** — build a self-contained piece of logic once, drop it into many parents. Multi-agent systems are the headline use (each agent a subgraph), but subgraphs are equally for plain modularity: a reusable "research" pipeline, a shared "validate-and-clean" routine.

```
sub_builder = StateGraph(SubState)
# ... build the subgraph ...
subgraph = sub_builder.compile()

parent_builder = StateGraph(ParentState)
parent_builder.add_node("do_subtask", subgraph)  # a compiled graph as a node
```

The key question: shared keys vs. transformed state

Subgraphs come in two flavors, and choosing wrong causes silent data loss:

- **Shared state keys.** Parent and subgraph share a state schema (or overlapping channels). The subgraph reads and writes the parent's channels directly. Simplest when both operate on the same data (e.g., both use `MessagesState`).
- **Different schemas with transformation.** The subgraph has its *own* state shape. You add the subgraph by wrapping it in a node function that **transforms** parent state → subgraph input, invokes the subgraph, and maps its output back. Use this when the subgraph is a reusable component with its own interface that shouldn't be coupled to the parent's schema.

```
def call_subgraph(state: ParentState) -> dict:
    sub_input = {"sub_field": state["parent_field"]}  # transform in
    sub_result = subgraph.invoke(sub_input)
    return {"parent_field": sub_result["sub_output"]}  # transform out
```

The intuition: if the parent and subgraph naturally speak the same state language, share keys; if the subgraph is a reusable black box with its own contract, transform at the boundary.

Streaming and persistence through subgraphs

Subgraphs participate in the parent's machinery, with a couple of wrinkles:

- **Streaming:** to receive events from inside subgraphs, set `subgraphs=True` on the stream call; otherwise you only see the parent's top-level events. `astream_events` (v3) surfaces nested subgraph events with namespacing so you can tell which subgraph a token came from.
- **Persistence:** the parent's checkpointer covers the whole tree — you compile the *parent* with a checkpointer, and subgraph state is checkpointed as part of it. You don't (and shouldn't) compile subgraphs with their own separate checkpointers when they're embedded.

Edge case — channels don't auto-propagate across differing schemas. If your subgraph writes a channel the parent's schema doesn't declare (and you used the transformation pattern), that data does not magically appear in parent state — you must map it out in the wrapper node. Silent "where did my subgraph's output go?" bugs are almost always an un-mapped channel at the boundary.

Edge case — interrupts inside subgraphs surface to the top. An `interrupt()` deep in a subgraph pauses the entire tree, and you resume from the top-level graph with the same `thread_id`. The pending interrupt's payload identifies where it came from. Resuming the subgraph directly (instead of via the parent) is a mistake — resume the parent.

Edge case — naming collisions. Two subgraphs that use the same node names are fine in isolation (LangGraph namespaces them), but when you're reading traces or routing with `Command`, be precise about which namespace you mean. Distinct, descriptive node names save hours of trace-reading.

Part IV — Edge Cases, Reliability & Production

The difference between a demo and a deployable system. These chapters concentrate the failure modes, the durability features, and the operational discipline that separate agents you can trust from agents that work on your machine.

20. Reducer & State Edge Cases

State is where most subtle bugs live, because the merge behavior is implicit. Here is the concentrated list.

`InvalidUpdateError` — *two parallel nodes wrote the same channel without a reducer.*

The canonical state bug. Either the two nodes shouldn't both write that channel, or the channel needs an appending reducer. The error is the BSP model refusing to pick a winner. Fix the design (who owns this channel?) before reaching for a reducer reflexively.

The disappearing update — last-write-wins clobbered your data. A channel with no reducer keeps only the most recent write. If you expected two nodes to both contribute and only see the second's data, you wanted an appending reducer and didn't declare one. Symptom: "only half my results are there."

In-place mutation isn't tracked. `state["items"].append(x)` and returning nothing may not register as a change, may skip reducers, and breaks time travel. Always return `{"items": [x]}` and let the reducer merge. Treat incoming state as immutable.

Returning the whole state clobbers others' keys. A node that returns the entire `state` dict (rather than just its changes) can overwrite keys other nodes own with stale values it read at super-step start. Return only what you changed.

Message duplication from unstable IDs. `add_messages` de-dupes by `id`. Messages without stable IDs that pass through a re-run can duplicate. Assign IDs when you need replace-not-append, and use `RemoveMessage` to delete.

Reducer crashes on the empty/initial value. Custom reducers must handle the channel's starting state (often `[]` or `None`). A reducer that assumes a non-empty current value fails on the first write. Guard the base case.

Non-deterministic order under parallel reduction. When parallel branches reduce into one channel, the merge order isn't guaranteed to match the diagram. If correctness depends on order, carry an index and sort downstream, or collect into per-branch keys and merge where you control the sequence.

State bloat from accumulating channels. An append-only channel (messages, logs, results) grows forever, ballooning every checkpoint. Trim, summarize, or window it (keep last N), and lean on `DeltaChannel` for long threads. Unbounded growth is a latency and cost leak, not just a memory one.

Big blobs in state. PDFs, images, huge JSON in a channel get re-serialized every checkpoint. Store the artifact externally; keep a reference in state.

Type drift between schema and reality. TypedDict doesn't validate at runtime, so a node can write a `str` to a field typed `int` and nothing complains until something downstream breaks. If inputs are untrusted, use a Pydantic schema to validate at the boundary.

The meta-lesson: **for every channel, know its merge rule and its growth.** Most state bugs are one of "wrong merge rule" or "unbounded growth."

21. Interrupt & Resume Edge Cases

Human-in-the-loop concentrates more sharp edges than any other feature. The full list, because each one is a production incident waiting to happen.

Side effects before `interrupt()` run twice. The defining edge case (Chapter 10): the node re-executes from the top on resume, so anything before the interrupt repeats. Pure computation before; side effects after. This causes double charges, double emails, duplicate records.

`try/except` swallows the interrupt. `interrupt()` raises `GraphInterrupt`; a broad `except` catches it and the pause silently never happens. Never wrap `interrupt()` in bare `except Exception`. Catch specific exceptions only.

`InMemorySaver` loses frozen threads on restart. A human takes an hour; you deploy in the meantime; the process restarts; every pending approval is gone. Production human-in-the-loop requires a durable checkpointer. This is the highest-severity interrupt mistake.

Wrong `thread_id` on resume. Resuming with a different `thread_id` than the initial call looks up the wrong (or empty) frozen state. The resume "does nothing" or starts fresh. Same `thread_id` for invoke and resume, always.

Multiple parallel interrupts with a single resume value. Fan-out where each branch interrupts leaves several pending interrupts; `Command(resume="x")` can't say which branch gets "x". Map by interrupt ID: `Command(resume={id_a: ..., id_b: ...})`.

The human never responds. Frozen threads accumulate forever. Build a TTL sweep: find threads older than your limit and auto-resolve, escalate, or re-notify.

Resuming a subgraph directly. An interrupt inside a subgraph pauses the whole tree; resume the parent with the shared `thread_id`, not the subgraph. Resuming the wrong level mishandles the frozen state.

Non-serializable interrupt payloads. The value passed to `interrupt()` is checkpointed, so it must be JSON-serializable. Passing a live object, a DB connection, or a non-serializable class breaks the pause. Pass plain data.

Resuming without inspecting the pending interrupt. Blindly resuming with a value before checking what was asked (and which interrupt is pending) leads to answering the wrong question. Read `get_state` / the `__interrupt__` payload first, then resume with the matching answer.

Assuming resume continues mid-node. It doesn't — it re-runs the node. Code that "knows" it's past the interrupt and skips earlier work is wrong; the earlier work re-runs. Design the node to be correct under full re-execution.

The unifying intuition: **resume = re-run the node with the answer pre-supplied.** Every interrupt edge case follows from that one fact plus "the frozen state must be durable and JSON-serializable."

22. Failure, Retries & Durable Execution

Production agents fail: models time out, tools error, pods die mid-run. LangGraph 1.2 turned durability and failure-handling into first-class features. This is the reliability toolkit.

Node retries with RetryPolicy

Wrap a node so transient failures retry automatically:

```
from langgraph.types import RetryPolicy

builder.add_node(
    "call_api", call_api,
    retry_policy=RetryPolicy(max_attempts=3, backoff_factor=2.0),
)
```

Use it for nodes that hit flaky external services. The intuition: retries belong on the *boundary* nodes (network, model, tool calls), not on pure-computation nodes (retrying deterministic code just repeats the same failure).

Per-node timeouts (1.2)

A node that hangs forever — a model call that never returns, an API with no timeout — can wedge an entire run. LangGraph 1.2 added **per-node timeouts**:

```
from langgraph.types import TimeoutPolicy

builder.add_node(
    "call_model", call_model,
    timeout=TimeoutPolicy(
        run_timeout=30.0,    # hard wall-clock cap for one attempt
        idle_timeout=10.0,  # cap on time with no progress; resets on activity
    ),
)
```

When a limit fires, LangGraph raises `NodeTimeoutError`, clears the writes from that attempt, and hands off to the retry policy. You can set a hard wall-clock limit, an idle limit that resets on progress, or both. **Async nodes only** — the timeout mechanism requires async execution. The intuition: `run_timeout` bounds total time; `idle_timeout` distinguishes "slow but working" from "stuck."

Node-level error handlers and the Saga pattern (1.2)

Retries handle *transient* failures. For *permanent* ones, 1.2 added **node-level error handlers** that run after all retries are exhausted:

```
def on_payment_failure(error) -> Command:
    # error is a typed NodeError; run compensation and route
```



```
refund_any_partial_charge()
return Command(goto="rollback", update={"status": "failed"})

builder.add_node("charge", charge_node, error_handler=on_payment_failure)
```

The handler receives a typed `NodeError` and returns a `Command` to update state and route elsewhere — exactly what you need for **Saga / compensation** patterns: when step 3 of a multi-step transaction fails permanently, run compensating actions to undo steps 1–2 and route to a recovery path, rather than crashing with a half-done transaction. The intuition: retries say "try again"; error handlers say "give up gracefully and clean up."

Graceful shutdown (1.2)

When you need to stop a running graph cooperatively — a deploy, a scale-down — without losing in-flight work, 1.2 added **graceful drain**:

```
from langgraph.types import RunControl

control = RunControl()
# ... pass control into the run; from any thread:
control.request_drain() # stop after the current super-step completes
```

The run stops after the current super-step finishes (not mid-node), raises `GraphDrained`, and saves a **resumable checkpoint** — so it can be picked up later with the same config. The intuition: drain is a *clean* stop at a checkpoint boundary, the opposite of killing the process and losing the super-step in flight.

The boundary problem: graph durability vs. workflow durability

A critical production realization: **LangGraph's checkpointer persists graph-internal state, but the graph usually sits inside a larger workflow** (load ticket → run graph → write report → notify). The checkpointer covers the graph; it does *not* cover the outer steps. The graph can finish perfectly and the *next* step (the report write, the notification) can fail.

Edge case — durable graph, non-durable surroundings. Don't assume that because the graph is checkpointed, the whole operation is durable. The outer workflow needs its own durability (a durable task queue, a workflow engine, idempotent outer steps). And `InMemorySaver` is not restart-durable at all — for graph resume across process/pod loss, you need a persistent checkpointer (Postgres/Redis/SQLite). Match each layer's durability to its failure modes.

The reliability mental model, in one line: **retries for transient, timeouts for stuck, error handlers for permanent, drain for clean stops, durable checkpointer for crashes — and remember the graph is only one layer of the system.**

23. Performance & Cost

Agents are expensive — every super-step may be an LLM call, and checkpoints have I/O cost. Performance work in LangGraph is mostly about **doing less model work and persisting less data**.

Checkpoint overhead and DeltaChannel

Every super-step writes a checkpoint. For long threads, naive full-state serialization makes each write grow with conversation length. `DeltaChannel` (1.2, beta) stores only the per-step delta, keeping checkpoint size flat as threads grow. For any long-running conversational agent, this is the single biggest persistence win. Pair it with message trimming/summarization so the *logical* state stays bounded too.

Node-level caching

Re-running an expensive, deterministic node wastes time and money. Cache it:

```
from langgraph.types import CachePolicy
from langgraph.cache.memory import InMemoryCache

builder.add_node("expensive", expensive_node, cache_policy=CachePolicy(ttl=300))
graph = builder.compile(cache=InMemoryCache())
```

Identical inputs within the TTL return the cached result (the output is even tagged `{"cached": True}`). The intuition: cache **pure, expensive** nodes (embedding computation, deterministic transforms); never cache nodes with side effects or time-dependent output.

Context window management is cost management

The dominant cost in a long agent run is **tokens**, and tokens scale with context length. Three levers:

- `SummarizationMiddleware` — condense old turns automatically before they bloat the prompt.
- **Message trimming/windowing** — keep only the last N messages, or the system prompt plus a window, dropping the stale middle.
- `LLMToolSelectorMiddleware` — don't send fifty tool schemas every call; select the relevant few.

***Edge case — accumulating tool outputs silently 10× your bill.** Each tool result stays in the message history and is re-sent every subsequent turn. A few verbose tool calls and your per-turn token count quietly explodes. Truncate/summarize tool outputs and prune old ones.*

Parallelism, with a leash

Fan-out (`Send` , parallel edges) cuts wall-clock time by running branches concurrently. But N parallel LLM calls is $N\times$ the instantaneous cost and $N\times$ the rate-limit pressure. Bound concurrency, batch work per worker, and remember that parallelism trades money and rate limits for latency. Use it where latency matters and the branch count is bounded.

Model selection per step

Not every node needs your most capable (most expensive) model. Use middleware (`wrap_model_call`) or per-node model binding to route simple steps (classification, routing, extraction) to a cheaper/faster model and reserve the flagship for hard reasoning. The intuition: **match the model to the difficulty of the step**. A routing decision doesn't need a frontier model.

Reliability features double as performance features

- `ModelFallbackMiddleware` keeps latency stable when the primary model degrades.
- **Prompt caching** (`AnthropicPromptCachingMiddleware`) reuses cached prefixes across turns, cutting both cost and time on multi-turn chats.
- **Streaming** doesn't reduce cost but transforms *perceived* latency — surface tokens as they generate.

The performance mental model: **the two scarce resources are tokens and checkpoint I/O. Reduce tokens (summarize, trim, select tools, cheaper models, caching) and reduce checkpoint size (DeltaChannel, no blobs, bounded channels). Parallelism buys latency at the cost of money.**

24. Recursion, Loops & Termination

Cyclic graphs can run forever; making them *stop correctly* is a core skill, and "stop" has more failure modes than it first appears.

The recursion limit, precisely

Every invocation has a `recursion_limit` (default 25) — the max super-steps before a `GraphRecursionError`. Recall it counts **super-steps, not loop iterations**: one `model → tools` round trip is two super-steps, so the default permits roughly a dozen tool-using turns.

```
graph.invoke(input, {"recursion_limit": 50})
```

Edge case — raising the limit to "fix" a loop hides the real bug. An agent that hits the recursion limit is usually oscillating: calling the same tool with the same arguments, or two nodes ping-ponging. Bumping the limit lets it oscillate longer and cost more. Diagnose why it loops before raising the ceiling. The limit is a seatbelt, not a parameter to tune away.

Designing real stopping conditions

A good agent terminates because it *decided to*, not because it crashed into the limit. The tools:

- **Tool-call / model-call budgets.** `ToolCallLimitMiddleware` and `ModelCallLimitMiddleware` enforce hard caps mid-run, short-circuiting to a final answer when hit. This is the clean, explicit budget the recursion limit only approximates.
- **An explicit "you must answer now" nudge.** When a budget is nearly exhausted, inject a system message instructing the model to give its best final answer without more tools. Far better UX than a raw error.
- **Loop detection.** Track recent tool calls in state; if the agent repeats an identical call, break the loop (route to a "stuck" handler or force a final answer).
- **Progress requirements.** For multi-step tasks, require measurable progress (a `ToDoList` advancing); if no item completes over several steps, intervene.

Termination edge cases

Edge case — the agent declares done prematurely. The opposite failure: the model answers before actually finishing the task (skips a required tool, hallucinates a result). Guard with deterministic post-checks — a fact-check node, a completeness validator — that can route back if the "final" answer is inadequate. Don't trust "no tool calls" as proof of completion for high-stakes tasks.

Edge case — infinite human-in-the-loop. A graph that interrupts, resumes, and interrupts again with no progress is an infinite loop with a human in it. Bound the number of clarification rounds.

Edge case — recursion limit inside a fan-out. Each `Send` branch consumes super-steps; a wide fan-out where each branch loops can hit the limit collectively. Account for parallel branches in your budget.

The intuition: **termination is a design responsibility, not a default.** Decide, before shipping, exactly how this agent stops — by budget, by validated completion, by loop detection — and make hitting the raw recursion limit an alarm, not normal operation.

25. Debugging & Observability

You cannot fix what you cannot see, and agents are unusually opaque — the "logic" lives partly in model outputs you didn't write. Observability is therefore not optional.

LangSmith: tracing built for agents

LangSmith is the companion platform for tracing every step of a LangGraph run: each node, each model call (prompt and response), each tool call (args and result), latency, token counts, and errors. For agent debugging it's transformative — instead of guessing why the model chose a tool, you read the exact prompt it saw and the exact response it gave. Enable it with environment variables and your runs are traced automatically. The intuition: **agent debugging is mostly "what did the model actually see, and what did it actually say?"** — and a tracer answers that directly.

Stream-based and state-based inspection

Without external tooling, LangGraph still exposes plenty:

- `stream_mode="debug"` emits maximally verbose events — every node entry/exit and state write — for live inspection during development.
- `stream_mode="updates"` shows exactly what each node changed, turn by turn — the fastest way to see *where* a run went wrong.
- `get_state` / `get_state_history` let you inspect the current and all past checkpoints — the state-level "what happened" (Chapter 9).
- `verbose=True` on `create_agent` prints node executions, state updates, and transitions — handy for debugging middleware behavior and agent flow.

Visualizing the graph

A graph you can see is a graph you can reason about:

```
graph.get_graph().draw_mermaid_png()    # or .draw_mermaid() for text
print(graph.get_graph().draw_ascii())
```

Rendering the topology catches structural bugs immediately — a missing back-edge, an unreachable node, a node with no exit. Make this a habit when building custom graphs.

Decoding the common errors

A quick field guide to the messages you'll actually see:

- `InvalidUpdateError` → two parallel writes to one channel without a reducer (Chapter 20).
- `GraphRecursionError` → hit the super-step limit; almost always an oscillating loop (Chapter 24).

- `GraphInterrupt` appearing where you didn't expect → an `interrupt()` fired; if it surfaced as an error, you probably caught it in a `try/except` (Chapter 21).
- `NodeTimeoutError` → a node exceeded its `TimeoutPolicy` (Chapter 22).
- "agent forgot everything" → wrong/changed `thread_id`, or `InMemorySaver` across a restart (Chapters 8, 21).
- `KeyError` on a state channel → a node read a channel that hasn't been written yet; use `.get()` with a default or declare an initial value.

Edge case — non-determinism makes bugs hard to reproduce. The same input can yield different runs because model outputs vary. Time travel (Chapter 9) is your friend: capture the failing thread, then replay from a checkpoint to reproduce deterministically up to the model call, and inspect from there. For tests, mock or pin the model (Chapter 26).

26. Testing Agents

Agents resist testing because the core component (the LLM) is stochastic and slow. The discipline is to **make everything around the model deterministic and testable, and to test the model's role separately**.

Test nodes in isolation

A node is a plain function `state -> dict`. Test it like one — no graph, no model required if the node is pure:

```
def test_router_finishes_when_no_tool_calls():
    state = {"messages": [AIMessage("Final answer", tool_calls=[])]}
    assert should_continue(state) == END
```

Routing functions, reducers, state transforms, and tool functions are all ordinary Python and should have ordinary unit tests. The majority of your *logic* bugs live here, not in the model, and these tests are fast and deterministic.

Mock or emulate the model

For tests that exercise the loop, replace the real model so tests are fast and deterministic:

- **A fake chat model** that returns scripted responses (a canned tool call, then a canned final answer) lets you test the full graph's control flow without API calls.
- `LLMToolEmulator` middleware emulates tools with an LLM (or canned behavior) so you can test an agent's tool-using behavior without hitting real APIs.

The intuition: **test the control flow with a scripted model** (does the graph loop, route, and terminate correctly given these model outputs?), and **evaluate the model's judgment separately** with evals.

Test the durability features

The features that matter most in production need their own tests:

- **Interrupts/resume:** drive a graph to an interrupt, assert the pending interrupt payload, resume with a value, assert the outcome — and specifically assert that **side effects before the interrupt don't double-fire** on resume (use a mock with a call counter).
- **Persistence:** run a thread, create a *new* graph instance with the same checkpointer and `thread_id`, and assert it resumes correctly — this catches "works in one process, forgets across instances" bugs.
- **Error handling:** force a node to fail and assert the `error_handler` / retry behaves (compensation runs, state routes correctly).

Evaluation, not just assertion

For the model-dependent behavior, unit assertions don't fit — you need **evals**: run the agent over a dataset of inputs and score outputs (with rubrics, reference answers, or an LLM judge) for quality, correctness, and regressions. LangSmith provides eval tooling for exactly this. The mindset shift: deterministic code gets *tests* (pass/fail); model behavior gets *evals* (scored over a set, tracked over time). Treat a drop in eval score on a prompt or model change as a regression, the same way you'd treat a failing test.

Edge case — time travel for reproducing flaky failures. When a bug only appears with a specific model output, capture the failing thread and replay from the checkpoint just before the divergence. This reproduces the exact state deterministically up to the model call, letting you debug a "non-reproducible" failure.

27. Common Bugs & Anti-Patterns

A consolidated field guide — the mistakes that recur across real LangGraph codebases. Skim this before shipping.

State and reducers - Missing reducer on a parallel-written channel → `InvalidUpdateError`.

Declare an appending reducer or rethink channel ownership. - **Mutating state in place** instead of returning a partial update — breaks tracking, reducers, and time travel. - **Returning the whole state** and clobbering keys other nodes own. Return only your changes. - **Unbounded message/list growth** quietly inflating latency and cost. Trim, summarize, window. - **Large blobs in state** re-serialized every checkpoint. Store externally, reference by key.

Interrupts and human-in-the-loop - Side effects before `interrupt()` firing twice on resume — the classic double-charge. Side effects after, always. - **`try/except` swallowing `GraphInterrupt`** so the pause never happens. - **`InMemorySaver` in production** losing frozen threads (and all memory) on restart. - **Changed `thread_id` between turns** → "the agent forgot everything." - **No TTL for abandoned interrupts** → frozen threads pile up forever.

Control flow - Missing loop-back edge in a custom agent → it calls one tool and halts. - **Raising the recursion limit to mask an oscillating loop** instead of fixing the loop. - **Trusting "no tool calls" as task completion** for high-stakes work — add deterministic post-checks. - **`goto` without `Command.PARENT`** in subgraph handoffs → routes into the wrong namespace and fails. - **Handoff loops** between peer agents with no budget.

Async and execution - Blocking calls in async nodes. A synchronous `requests.get` or `time.sleep` inside an async node blocks the event loop and serializes everything. Use async clients (`httpx`, `aiohttp`) in async nodes, or keep the node sync. Per-node timeouts (1.2) require async nodes — another reason to get this right. - **Assuming parallel nodes see each other's writes.** They read start-of-super-step state; cross-branch reads are a logic error.

Agents and tools - Fifty tools on one agent, confusing it. Split into specialists, or use `LLMToolSelectorMiddleware`. - **Vague tool descriptions** causing wrong/missing calls. The docstring is the model's spec — write it precisely. - **Silently swallowing tool errors** so the model loops trying to fix what it can't see. Decide deliberately between surfacing the error to the model and failing loudly. - **Non-idempotent side-effecting tools** under retries/replays/interrupts → duplicate actions.

Architecture - Reaching for a fully autonomous agent where a workflow fits. The most expensive anti-pattern of all: handing the LLM control of the whole flow when most of it is deterministic. Constrain the agent to the steps that genuinely need judgment. -

`HumanInTheLoopMiddleware` without a checkpointer → crashes instead of pausing. - **Confusing the store with the checkpointer** → ephemeral junk in long-term memory, or durable facts lost when a thread ends.

The pattern across nearly all of these: **a missing rule for an implicit behavior** (how a channel merges, what re-runs on resume, who owns control flow, what's durable). Make the implicit explicit and the bug class disappears.

28. Best Practices & Mental Models Reference

A distilled reference — the intuitions and decision rules from the whole book in one place.

The core proverbs

- **An agent run is a durable state machine.** If you forget everything else, keep this.
- **Use the highest-level tool that fits; drop down only when you must.** `create_agent` → middleware → custom `StateGraph`.
- **Give the LLM authority only where judgment is genuinely needed.** Everything else should be deterministic.
- **For every channel, know its merge rule and its growth.** Most state bugs are wrong-merge or unbounded-growth.
- **Resume means re-run the node with the answer pre-supplied.** Every interrupt edge case follows from this.
- **Pure computation before `interrupt()` ; side effects after.**
- **The two scarce resources are tokens and checkpoint I/O.** Optimize both.
- **Termination is a design responsibility, not a default.**
- **Make the implicit explicit** — the merge rule, the budget, the durability, the control owner.

Decision: agent vs. workflow

Ask *who should decide the control flow*. If you can enumerate the steps at design time → **workflow** (fixed edges, cheap, testable). If the path genuinely depends on the model's judgment → **agent** (LLM-routed). Most real systems are a **workflow with agentic nodes** — place the dial deliberately, per transition.

Decision: prebuilt vs. custom vs. functional

- `create_agent` — a single tool-calling agent, possibly with structured output and memory. Start here.
- **Custom** `StateGraph` — you need to shape the control flow: deterministic steps between agentic ones, bespoke routing, multiple sub-agents, rich state.
- **Functional API** (`@entrypoint` / `@task`) — the logic is naturally imperative (`if` / `for` / `while`) and you want durability without drawing a graph.

Decision: checkpointer vs. store

- **Checkpoint** — short-term, thread-scoped memory: *what happened in this conversation*. Required for interrupts and resume.

- **Store** — long-term, cross-thread memory: *what I know that outlives any conversation*. Namespaced, optionally semantic.

Match the data's lifetime to the mechanism; never mix them.

Decision: conditional edges vs. Command

- **Conditional edges** — routing is a pure function of resulting state; keeps topology explicit. Default.
- **Command** — routing is a byproduct of the node's own computation, or you're doing agent handoff. Use `Command.PARENT` to navigate the parent graph.

Decision: which interrupt

- **Dynamic** `interrupt()` — conditional pause, rich payload, structured input. Modern default.
- **Static** `interrupt_before/after` — blanket "always pause here" policy or simple debugging gate.

The pre-production checklist

Before an agent goes live, verify every line:

1. **Durable checkpointer** (Postgres/Redis/SQLite), never `InMemorySaver`, anywhere a process can restart or a human can pause.
2. `thread_id` **is stable** per conversation and correct on every resume.
3. **Side effects are after** `interrupt()` and idempotent under retry/replay.
4. **No** `try/except` **swallowing** `GraphInterrupt`.
5. **Explicit termination** — a tool/model-call budget, validated completion, loop detection — not reliance on the raw recursion limit.
6. **Bounded context** — summarization or windowing on message history; tool outputs trimmed; no blobs in state.
7. **Reliability middleware** — model fallback, retries (model and tool), call limits — composed in a deliberate order.
8. **Failure handling** — `RetryPolicy` for transient, `TimeoutPolicy` for stuck (async nodes), `error_handler` / Saga for permanent.
9. **Graceful drain** wired for clean shutdowns; the *outer* workflow has its own durability.
10. **Observability** — LangSmith tracing on; you can answer "what did the model see and say?" for any run.
11. **PII handling** if outputs are logged or displayed.
12. **Tests for logic** (nodes, routers, reducers) and **evals for model behavior**, including an explicit test that interrupt side effects don't double-fire.

The toolchain

- `pip install -U langgraph langchain langchain-openai langchain-anthropic` — runtime, agents, providers.
 - `langgraph-checkpoint-postgres` (or `sqlite/redis`) — durable persistence for production.
 - **LangSmith** — tracing and evaluation.
 - **LangGraph Studio / Platform** — visual debugging and deployment for graphs.
 - `graph.get_graph().draw_mermaid_png()` — visualize topology while building.
-

Closing: The Path to Mastery

You now have the whole map. The journey from here is deliberate practice against the three things you asked for.

Intuition comes from holding the core model — *a durable state machine* — and re-deriving everything else from it. Why do parallel writes need a reducer? Because super-steps apply updates together. Why does resume re-run the node? Because checkpoints are per-super-step. Why is human-in-the-loop a persistence feature? Because pausing for days means walking away from a frozen, durable state. When a new feature appears, ask "how does this serve the durable state machine?" and it will slot into place.

The "ways" become second nature by building the same thing three times: once with `create_agent`, once as a custom `StateGraph`, once with the functional API. Feel where each is comfortable and where each strains. Do the same for routing (conditional edge vs. `Command`), for memory (checkpointer vs. store), for pausing (static vs. dynamic interrupt). Mastery isn't knowing one way; it's knowing which way a given problem deserves.

The edge cases are learned by respecting them before they bite. Re-read Part IV before each deploy. Wire the double-charge test. Switch off `InMemorySaver`. Set the budget. Each edge case you internalize is an incident that never happens.

A suggested path: build a single-agent assistant with `create_agent` and a checkpointer; add tools, then human-in-the-loop for a sensitive one; add summarization and a call budget; rebuild its core as a custom graph to feel the control; then compose two specialists into a supervisor. Trace every run in LangSmith. Deliberately trigger each Part IV edge case in development so you recognize it instantly in production.

The framework will keep moving — new middleware, new channel types, new durability features. But the foundation you now hold is stable: **state, reducers, control flow, persistence, the agent loop, and the discipline of edge cases**. Build something real, break it on purpose, fix it well. That is the path to mastery.